



UNIVERSITÀ DEGLI STUDI DI SALERNO

Dipartimento di Informatica

Corso di Laurea Triennale in Informatica

TESI DI LAUREA

Titolo della Tesi

RELATORE

Prof. Fabio Palomba

Dott. Nome tutor

Università degli Studi di Salerno

CANDIDATO

Marco Donatiello

Matricola: 0512119045

Anno Accademico 2025-2026

Questa tesi è stata realizzata nel

sesa^{lab}
SOFTWARE ENGINEERING
SALERNO

Dedica o citazione

Abstract

Il testo dell'abstract qui...

Indice

Elenco delle Figure	iv
Elenco delle Tabelle	v
1 Introduzione	1
2 Analisi dello stato del arte e contesto tecnologico	2
2.1 OWASP Cloud Security Top 10	2
2.1.1 IaC misconfiguration	4
2.2 API Security Risks	6
2.2.1 API1:2023 – Broken Object Level Authorization	8
2.2.2 API2:2023 – Broken Authentication	15
2.2.3 API3:2023 – Broken Object Property Level Authorization	17
2.2.4 API4:2023 – Unrestricted Resource Consumption	20
2.2.5 API5:2023 – Broken Function Level Authorization	22
2.2.6 API6:2023 – Unrestricted Access to Sensitive Business Flows	25
2.2.7 API7:2023 – Server Side Request Forgery	27
2.2.8 API8:2023 – Security Misconfiguration	29
2.2.9 API9:2023 – Improper Inventory Management	31
2.2.10 API10:2023 – Unsafe Consumption of APIs	33

3	Metodo di ricerca	36
3.1	Approccio metodologico	36
3.1.1	Obiettivi e domande di ricerca	36
3.1.2	Impostazione dello studio	37
3.1.3	Il criterio di classificazione	38
3.1.4	Requisiti derivati per lo strumento di rilevazione	39
3.1.5	Disegno della valutazione	40
3.2	Rilevamento delle IaC misconfiguration	41
3.2.1	Architettura del sistema	41
3.2.2	Lo scanner Checkov e la traduzione nel modello di dominio .	44
3.2.3	Correlazione e punteggio di rischio	46
3.2.4	Validazione e limiti	49
3.3	Rilevamento della Broken Authentication	51
3.3.1	Ricostruzione del modello di autenticazione	51
3.3.2	La verifica dinamica	53
3.3.3	Il caso senza sessione: la regola statica S01	54
3.3.4	Validazione e limiti	55
3.4	Rilevamento della Broken Object Property Level Authorization . . .	56
3.4.1	Inventario delle proprietà	57
3.4.2	Inferenza dell'autorizzazione per proprietà	58
3.4.3	La verifica dinamica per proprietà	59
3.4.4	Validazione e limiti	60
3.5	Rilevamento dell'Unrestricted Resource Consumption	61
3.5.1	I tre livelli di analisi	61
3.5.2	Aggregazione e correlazione fra livelli	63
3.5.3	Validazione e limiti	63
3.6	Rilevamento della Broken Function Level Authorization	65
3.6.1	I tre livelli e la mappatura degli endpoint	66
3.6.2	Deduplicazione per priorità di regola	67
3.6.3	Validazione e limiti	68
3.7	Rilevamento della Server-Side Request Forgery	69
3.7.1	Metodo di ricerca e sistema di rilevazione	69

3.8	Rilevamento della Security Misconfiguration	72
3.8.1	Regole locali e regole globali	72
3.8.2	Limiti del modulo	74
3.9	Rilevamento dell'Unsafe Consumption of APIs	74
3.9.1	Le tre regole	75
3.9.2	Validazione e limiti	76
4	Conclusioni	77
	Bibliografia	78

Elenco delle figure

3.1	Diagramma dei componenti del sistema	42
3.2	Diagramma delle classi del dominio e del motore di correlazione . .	43
3.3	Diagramma di attività dell'adapter Checkov	47
3.4	Diagramma di sequenza della pipeline statica	48

Elenco delle tabelle

3.1	Policy Checkov pertinenti	45
3.2	Risoluzione dei controlli nel catalogo delle policy	46
3.3	Punteggi di rischio sul bersaglio Terraform	50
3.4	Batteria di prove dinamiche della Broken Authentication	53
3.5	Riferimento di verità per la Broken Authentication	55
3.6	Batteria di prove dinamiche della BOPLA	59
3.7	Regole del modulo Unrestricted Resource Consumption	64
3.8	Regole del modulo Broken Function Level Authorization	67
3.9	Regole del modulo Security Misconfiguration	73
3.10	Regole del modulo Unsafe Consumption of APIs	75

CAPITOLO 1

Introduzione

Alcune linee guida per scrivere la tesi: 1) Ogni immagine/tabella deve avere le seguenti caratteristiche: a) Una caption; b) Deve essere esplicitamente commentata all'interno del testo (ricordatevi che se vi riferite alla figura/tabella inserendo anche il corrispettivo numero e.g., Figura X mostra che... allora la prima lettera deve essere scritta in maiuscolo; E ricordatevi che latex mette a disposizione un comando per citare correttamente (ref) senza dover mettere a mano il numeretto corrispondente) c) Le immagini devono essere rifatte tramite PPT o similari e salvate in pdf prima di inserirle nel file latex, altrimenti rischiate di inserire immagini sgranate. 2) Ricordatevi sempre di citare le fonti e di rifrasare, altrimenti rischiate di incorrere nel reato di plagio (che può comportare anche la revoca del titolo accademico) e che citare non sminuisce il vostro lavoro, anzi, lo rafforza. 3) L'Abstract deve necessariamente contenere le seguenti informazioni a) Un breve riassunto della problematica; b) Una breve descrizione di ciò che è stato fatto; c) I risultati ottenuti.

Analisi dello stato del arte e contesto tecnologico

2.1 OWASP Cloud Security Top 10

Il portale AppSecMaster ha elaborato una propria classificazione dei dieci rischi più comuni e di maggiore impatto nei moderni ambienti cloud, ispirandosi ai principi promossi da OWASP in materia di sicurezza applicativa, che vanno dall'infrastruttura mal configurata alla scarsa visibilità delle risorse in esecuzione. Di seguito è riportata la lista delle principali problematiche individuate:

1. **Security Misconfiguration** — Bucket di storage lasciati con permessi troppo permissivi o istanze di calcolo distribuite senza le dovute restrizioni restano tra le cause più frequenti di violazione dei dati in ambienti cloud.
2. **Identity / Access Mismanagement** — Ruoli IAM assegnati con privilegi superiori a quelli effettivamente necessari, e mai rivisti dopo la configurazione iniziale, espongono i sistemi a un concreto rischio di escalation dei permessi.
3. **Improper Secrets/Key Management** - Se chiavi API e credenziali vengono salvate in chiaro o incorporate nel codice sorgente, la loro compromissione può aprire un accesso laterale a più servizi collegati.

4. **Insecure Network Segmentation** — Senza una reale separazione tra reti e ambienti diversi, un attaccante che viola un singolo componente può muoversi liberamente verso risorse più sensibili.
5. **Injection / Input Validation Flaws in Cloud Services** — Le classiche vulnerabilità di injection assumono forme nuove nei contesti cloud-native, colpendo funzioni serverless, code di messaggi e gateway API dove i controlli sugli input non sono sempre applicati in modo uniforme.
6. **Insecure Deserialization / API Exposure** — La gestione non sicura di dati serializzati, unita ad API interne prive di adeguati controlli di autenticazione o versionamento, costituisce un punto d'ingresso sfruttabile per eseguire codice non autorizzato.
7. **Vulnerable Third-Party/Container Dependencies** — Librerie open source e immagini container non aggiornate accumulano vulnerabilità note, soprattutto quando le stesse basi vengono riutilizzate tra più progetti senza verifiche periodiche.
8. **Improper Asset Inventory / Visibility** — Risorse cloud dimenticate o mai tracciate correttamente, incluse istanze abbandonate e servizi "ombra", sfuggono facilmente al controllo dei team di sicurezza.
9. **Insufficient Logging / Monitoring** — Log distribuiti su decine di servizi diversi e privi di un formato coerente rendono difficile individuare per tempo un'intrusione in corso.
10. **Unvalidated Redirects/Cloud Function Abuse** — Funzioni serverless eseguite con permessi troppo ampi, combinate a controlli insufficienti sui redirect, possono essere sfruttate per aggirare le restrizioni di accesso e ottenere un punto d'appoggio nascosto nell'infrastruttura.

Tra le problematiche elencate, il presente lavoro si concentra sulla prima, indicata nel seguito come IaC misconfiguration¹, e sulla componente di esposizione delle API della sesta, Insecure Deserialization / API Exposure: sono le due voci più direttamente legate agli obiettivi del sistema sviluppato e quelle che ricorrono con maggiore frequenza negli scenari di autorizzazione tipici delle architetture basate su API. La prima è approfondita nella sottosezione seguente; la seconda nella Sezione 2.2, dove la classificazione OWASP dedicata alle API la articola in dieci categorie distinte.

2.1.1 IaC misconfiguration

Una IaC misconfiguration nasce da un errore nella configurazione di un servizio, di un prodotto o di una risorsa infrastrutturale: le impostazioni di sicurezza predefinite non vengono irrobustite a sufficienza, e questo lascia scoperto un fianco che con poco sforzo potrebbe essere protetto. I numeri parlano chiaro: il documento ufficiale OWASP riporta che il 90% delle applicazioni testate è risultato vulnerabile ad almeno una forma di misconfiguration [1]. Nella maggior parte dei casi non c'è nulla di sofisticato dietro: è una dimenticanza, un'impostazione lasciata di default, un permesso che nessuno ha mai rivisto. Ed è proprio questo che la distingue da una vulnerabilità nel codice applicativo — qui il problema non sta nella logica del software, ma in come le risorse sono state predisposte.

Le forme più comuni in cui una misconfiguration si manifesta sono, in fondo, poche e ricorrenti. Un bucket di storage lasciato accessibile pubblicamente, quando avrebbe dovuto restare privato, è probabilmente la più diffusa: basta una policy impostata male, magari copiata da un altro progetto senza troppa attenzione. Un container che gira con privilegi di root, invece, espone il sistema a un rischio diverso ma altrettanto concreto — se l'attaccante riesce a comprometterlo, si ritrova con permessi ben oltre quelli che gli servirebbero per fare danni. C'è poi il caso dei template Infrastructure-as-Code scritti senza le dovute verifiche: un file Terraform o CloudFormation che definisce una risorsa con permessi troppo ampi si porta dietro

¹IaC (*Infrastructure as Code*): pratica per cui le risorse infrastrutturali — reti, istanze, storage, permessi — sono dichiarate in file versionati che uno strumento di provisioning traduce nello stato reale; il file diventa così un artefatto ispezionabile prima che la risorsa esista.

lo stesso errore ogni volta che viene riutilizzato, replicandolo su ambienti diversi senza che nessuno se ne accorga.

Un caso reale: il data breach Capital One Il caso Capital One del luglio 2019 resta probabilmente l'esempio più citato di questa categoria di rischio. Un bucket Amazon S3 lasciato con permessi troppo ampi permise a un'attaccante di sottrarre oltre 100 milioni di record clienti, tra dati anagrafici, indirizzi e informazioni finanziarie sensibili [2]. Nessuna falla nella piattaforma cloud era coinvolta: la responsabilità ricadeva interamente sulla configurazione impostata dal cliente. Non a caso AWS, commentando l'accaduto, ribadì che *"Amazon S3 is secure by default"* [3], scaricando di fatto la colpa sulla gestione dei permessi lato utente. Sul piano economico il conto fu salato: 80 milioni di dollari di multa dall'Office of the Comptroller of the Currency, più 190 milioni versati per chiudere una class action. Capital One dimostra bene quanto una semplice disattenzione in fase di configurazione possa costare quanto, se non più di, un attacco costruito su un exploit tecnicamente elaborato.

Episodi come questo spiegano perché, negli ultimi anni, le organizzazioni abbiano iniziato ad affidarsi sempre meno alla revisione manuale delle configurazioni e sempre più a strumenti automatizzati capaci di individuare questo tipo di errori prima che vengano sfruttati. Una configurazione insicura, del resto, raramente viene introdotta di proposito: emerge quasi sempre da un template riutilizzato senza controllo, da un default mai cambiato, da una revisione saltata per mancanza di tempo — fattori che un controllo manuale sistematico difficilmente riesce a intercettare su larga scala e con continuità.

Il documento ufficiale OWASP illustra questa categoria di rischio attraverso quattro scenari esemplificativi [1]. Il primo riguarda applicazioni di esempio dimenticate su un server di produzione, spesso accompagnate da account amministrativi con credenziali mai modificate rispetto ai valori predefiniti. Il secondo descrive un server su cui la funzionalità di directory listing resta abilitata, consentendo a un attaccante di risalire fino al codice sorgente tramite reverse engineering. Il terzo riguarda messaggi di errore troppo dettagliati restituiti dall'application server, capaci di esporre informazioni sull'infrastruttura sottostante o versioni di componenti vulnerabili.

Il quarto, infine, riguarda permessi di condivisione predefiniti di un cloud service provider lasciati aperti ad altri utenti su Internet, con conseguente esposizione di dati sensibili archiviati nello storage cloud.

Fra i quattro scenari, due ricadono nel perimetro del sistema presentato in questo lavoro e due ne restano fuori. Rientrano il secondo e il quarto, perché in entrambi la configurazione difettosa è dichiarata in un artefatto ispezionabile — la direttiva che abilita il directory listing nella configurazione del server, i permessi di condivisione fissati nel manifest della risorsa di storage — e può quindi essere individuata prima del rilascio. Restano fuori il primo e il terzo: le applicazioni di esempio dimenticate in produzione non compaiono in alcun artefatto del progetto e presuppongono un inventario degli host effettivamente in esecuzione, mentre i messaggi di errore troppo dettagliati emergono soltanto dalle risposte prodotte a runtime.

2.2 API Security Risks

Le API oggi fanno gran parte del lavoro che una volta faceva il sito web stesso: non restituiscono una pagina pensata per essere letta da una persona, ma dati grezzi destinati a un'altra app, a un altro servizio, a un sistema che poi decide cosa farne. Lo schema delle richieste è prevedibile, gli endpoint² espongono operazioni precise su risorse precise. Per un attaccante questo significa meno lavoro di interpretazione: non deve capire un'interfaccia grafica, gli basta osservare come sono costruite le chiamate.

OWASP mantiene, accanto al Top 10 per le applicazioni web, una classificazione separata dedicata alle API. Alcune categorie compaiono in entrambe le liste — la misconfiguration, il broken access control — ma buona parte dei rischi specifici delle API riguarda un livello che il Top 10 generale non copre: l'autorizzazione su singoli oggetti, proprietà, funzioni. Un'applicazione web tradizionale filtra cosa mostrare all'utente; un'API deve filtrare cosa restituire nella risposta. La differenza sembra minima, ma è da lì che nascono molte delle vulnerabilità trattate in questa sezione.

²*Endpoint*: il singolo punto di accesso di un'API, identificato da un URL e da un metodo HTTP, che espone una specifica operazione su una risorsa.

Le dieci categorie che compongono l'edizione 2023 riflettono questa logica: non rischi generici, ma rischi legati al modo specifico in cui un'API espone dati e funzionalità. Di seguito la lista completa, in ordine di gravità secondo OWASP [4]:

1. **API1:2023 – Broken Object Level Authorization (BOLA)** — un endpoint accetta l'ID di una risorsa senza verificare che chi lo richiede abbia davvero il diritto di accedervi; basta cambiare un numero nell'URL per leggere dati altrui.
2. **API2:2023 – Broken Authentication** — meccanismi di autenticazione implementati male, token che non scadono mai, o password recuperabili senza controlli sufficienti.
3. **API3:2023 – Broken Object Property Level Authorization** — qui il problema non è accedere alla risorsa sbagliata, ma vedere o modificare campi che non dovrebbero essere esposti o scrivibili da quel particolare utente.
4. **API4:2023 – Unrestricted Resource Consumption** — assenza di limiti su chiamate, dimensione dei payload o uso di risorse computazionali, con conseguenze che vanno dal semplice rallentamento a un vero e proprio denial of service.
5. **API5:2023 – Broken Function Level Authorization** — un utente normale riesce a invocare funzioni riservate agli amministratori, semplicemente perché il controllo sui permessi manca a livello di endpoint o di metodo HTTP.
6. **API6:2023 – Unrestricted Access to Sensitive Business Flows** — la singola richiesta può essere anche legittima, ma se ripetuta migliaia di volte automatizza un abuso della logica di business (es. acquisto massivo di biglietti, creazione automatica di account).
7. **API7:2023 – Server Side Request Forgery (SSRF)** — l'API accetta un URL fornito dall'utente e lo interroga senza validarlo, permettendo di raggiungere risorse interne altrimenti non accessibili dall'esterno.

8. **API8:2023 – Security Misconfiguration** — header di sicurezza mancanti, CORS³ troppo permissivo, ambienti di debug ancora attivi in produzione: stessa famiglia di errori vista nel capitolo precedente, qui applicata allo strato applicativo.
9. **API9:2023 – Improper Inventory Management** — endpoint vecchi, versioni deprecate o ambienti di staging dimenticati online, mai rimossi né monitorati, restano comunque raggiungibili da chi li trova.
10. **API10:2023 – Unsafe Consumption of APIs** — la vulnerabilità non nasce nell'API stessa, ma nella fiducia cieca riposta nei dati restituiti da API di terze parti integrate nel sistema.

Tra le dieci categorie appena elencate la prima merita un'attenzione diversa dalle altre: non tanto per la posizione in classifica, quanto per ciò che rappresenta, cioè un controllo di accesso che manca proprio dove ci si aspetterebbe fosse più solido, sulla singola risorsa richiesta. È il tipo di errore che si insinua facilmente in un endpoint scritto in fretta e che, allo stesso tempo, può esporre porzioni estese della base dati di un sistema con una richiesta HTTP modificata di poco. La sottosezione seguente lo esamina nel dettaglio: come si manifesta, da quali cause nasce e con quali misure lo si previene.

2.2.1 API1:2023 – Broken Object Level Authorization

Definizione e collocazione La Broken Object Level Authorization (BOLA) si manifesta quando un'applicazione, pur autenticando correttamente l'utente che effettua una richiesta, non verifica se quello stesso utente abbia effettivamente il diritto di accedere all'oggetto specifico richiesto [5].

Un'interfaccia di programmazione applicativa espone tipicamente un insieme di risorse — utenti, ordini, documenti, transazioni — identificabili singolarmente attraverso un riferimento univoco. L'accesso a tali risorse presuppone due verifiche

³CORS (*Cross-Origin Resource Sharing*): il meccanismo con cui un server dichiara quali siti di origine diversa possono leggere le sue risposte dal browser. Una politica permissiva le rende leggibili a qualunque origine.

concettualmente distinte, spesso confuse nella pratica implementativa: l'autenticazione, che accerta l'identità del soggetto che effettua la richiesta, e l'autorizzazione, che stabilisce se quel soggetto, una volta identificato, abbia il diritto di operare su una specifica risorsa.

Un utente autenticato correttamente non è, per definizione, un utente autorizzato ad accedere a qualunque oggetto gestito dal sistema. Questa distinzione, apparentemente elementare, è proprio il punto in cui molte implementazioni falliscono: il possesso di credenziali valide viene talvolta considerato — implicitamente, nel disegno del codice — condizione sufficiente per l'accesso a una risorsa, quando in realtà rappresenta soltanto la premessa per una verifica ulteriore, relativa alla relazione specifica tra quell'utente e quell'oggetto.

Funzionamento delle API e ruolo degli identificatori di risorsa Le API REST, e in misura diversa quelle basate su GraphQL⁴, espongono le risorse attraverso identificatori che compaiono nel percorso della richiesta, nei parametri di query, nel corpo del messaggio o negli header. Un endpoint come `GET /api/orders/482` restituisce, nella maggior parte delle implementazioni, l'oggetto ordine identificato dal valore 482, indipendentemente dal fatto che tale valore sia un intero progressivo, un UUID o un altro tipo di riferimento.

La gestione di una richiesta di questo tipo comporta, lato server, due passaggi distinti: la risoluzione dell'identificatore in un oggetto concreto, tramite interrogazione del database o di un servizio dati, e il controllo che l'utente autenticato abbia diritto di accesso a quell'oggetto specifico. Il primo passaggio è quasi sempre implementato correttamente, poiché è indispensabile al funzionamento stesso dell'endpoint; il secondo, non avendo un effetto visibile quando manca, viene più facilmente trascurato o dato per scontato in fase di sviluppo.

Va osservato che l'endpoint, in questo scenario, non è di per sé vietato all'utente: la sua configurazione di ruoli e permessi lo autorizza a invocare quella funzione API. Questo distingue concettualmente BOLA da un'altra categoria della stessa famiglia,

⁴GraphQL: linguaggio di interrogazione per API in cui il client, anziché richiamare endpoint predefiniti, descrive in una sola richiesta l'esatto insieme di dati che desidera ricevere.

la Broken Function Level Authorization (BFLA), in cui è l'accesso alla funzione stessa a non essere autorizzato. In BOLA la funzione è legittima; il problema riguarda esclusivamente quale istanza dell'oggetto venga restituita o modificata.

Meccanismo dell'attacco Lo sfruttamento di una vulnerabilità BOLA non richiede, nella maggior parte dei casi, strumenti sofisticati né la compromissione di credenziali altrui. È sufficiente disporre di un account legittimo sul sistema bersaglio. Il percorso tipico dell'attacco segue una sequenza riconoscibile: l'utente accede al proprio account, richiede una propria risorsa attraverso l'interfaccia normale dell'applicazione e osserva, tramite gli strumenti di sviluppo del browser o un proxy di intercettazione, l'identificatore associato a quella risorsa nella richiesta sottostante.

A questo punto l'identificatore viene sostituito con un altro valore plausibile per lo stesso schema — un intero vicino, nel caso di sequenze numeriche, oppure un valore ottenuto per enumerazione più sistematica quando lo schema lo consente — e la richiesta viene reinviata con lo stesso token di autenticazione, valido e non modificato. Ad esempio:

```
GET /api/orders/483 HTTP/1.1
Host: api.example.com
Authorization: Bearer <token_utente_A>
```

dove 483 non è l'ordine appartenente al titolare del token, ma un identificatore vicino a quello osservato in precedenza durante il normale utilizzo dell'applicazione. La richiesta è, dal punto di vista del protocollo, del tutto regolare: cambia soltanto il valore del parametro. Che la risposta contenga davvero i dati di un altro utente dipende da un unico fattore, ripreso nel paragrafo seguente: se il server confronti o meno l'identità del chiamante con il proprietario effettivo dell'oggetto.

Esempio applicativo Si consideri la risorsa `/api/orders/{id}`, utilizzata per recuperare i dettagli di un ordine. Una richiesta legittima, effettuata dal proprietario dell'ordine 482, ha la forma:

```
GET /api/orders/482 HTTP/1.1
Host: api.example.com
```

```
Authorization: Bearer <token_utente_A>
```

e produce una risposta simile alla seguente:

```
HTTP/1.1 200 OK
```

```
Content-Type: application/json
```

```
{
  "order_id": 482,
  "customer_id": 1041,
  "items": ["..."],
  "shipping_address": "..."
```

La richiesta modificata introdotta nel paragrafo precedente, diretta all'ordine 483 e appartenente a `customer_id` 1078, viene gestita da un'implementazione vulnerabile allo stesso modo: l'identificatore viene estratto dal percorso, la tabella corrispondente viene interrogata, il risultato viene restituito. In nessun punto di questo flusso il valore di `customer_id` restituito viene confrontato con l'identità associata al token presente nell'header `Authorization`. La risposta, di conseguenza, è ancora un 200 OK, questa volta con i dati di un ordine che non appartiene al chiamante.

Il punto critico non è la struttura della richiesta, identica nei due casi, ma l'assenza di un confronto esplicito fra utente autenticato e risorsa richiesta. Un'implementazione corretta dovrebbe, prima di restituire l'oggetto, verificare che il `customer_id` dell'ordine coincida con l'identità del chiamante — o che quest'ultimo disponga di un diritto di accesso esplicitamente concesso su quella risorsa — e restituire, in caso contrario, un errore di autorizzazione: tipicamente 403 `Forbidden`, oppure 404 `Not Found` quando si preferisce non rivelare l'esistenza della risorsa a chi non ne ha diritto.

Cause e meccanismi tecnici tipici L'assenza di un controllo di autorizzazione a livello di oggetto è la causa immediata di ogni istanza di BOLA, ma raramente costituisce una scelta progettuale deliberata. Più spesso deriva da alcune condizioni che meritano di essere analizzate singolarmente.

La prima riguarda la sovrapposizione concettuale tra autenticazione e autorizzazione nella fase di implementazione. Un endpoint protetto da un middleware di autenticazione, che verifica la validità del token e ne estrae l'identità, può apparire sicuro a chi lo sviluppa, inducendo a trascurare la necessità di un controllo ulteriore specifico per l'oggetto acceduto. Il middleware risponde alla domanda "chi sta chiamando?", non a "questo chiamante ha diritto su questo oggetto?".

Una seconda causa è la fiducia implicita nei parametri forniti dal client. Alcune implementazioni derivano l'identità del proprietario di una risorsa dallo stesso identificatore fornito nella richiesta, oppure presuppongono che, poiché l'interfaccia utente dell'applicazione non espone mai identificatori altrui, un client "onesto" non li invierebbe mai. Questa assunzione ignora che un client HTTP può essere costruito o modificato arbitrariamente da chi effettua la richiesta, indipendentemente dall'interfaccia grafica ufficiale.

Un terzo fattore è rappresentato da controlli di autorizzazione implementati in modo incompleto o incoerente tra i diversi endpoint di una stessa API. È relativamente comune riscontrare verifiche corrette sull'operazione di cancellazione di una risorsa e verifiche assenti sull'operazione di lettura o aggiornamento della medesima risorsa, poiché ciascun endpoint viene sviluppato e sottoposto a revisione in momenti diversi, con criteri non sempre uniformi. La crescita organica di un'API nel tempo, con l'aggiunta progressiva di nuovi endpoint da parte di sviluppatori diversi, tende ad amplificare questa disomogeneità.

Una causa più strutturale riguarda infine la gestione delle relazioni tra utenti e risorse nel modello dati. Quando tale relazione non è rappresentata in modo esplicito e interrogabile — ad esempio tramite una chiave esterna diretta tra la tabella degli utenti e quella delle risorse, o tramite un livello di autorizzazione centralizzato — ogni endpoint deve reimplementare autonomamente la logica di verifica, e questo aumenta la probabilità che qualche percorso venga trascurato.

Impatto L'impatto di una vulnerabilità BOLA dipende dalla natura della risorsa esposta, ma incide tipicamente su più proprietà di sicurezza contemporaneamente. Sul piano della riservatezza, l'accesso non autorizzato a un oggetto consente la lettura di dati che dovrebbero rimanere riservati al proprietario legittimo: informazioni

personali, dettagli finanziari, contenuti sanitari, comunicazioni private, a seconda del dominio applicativo.

Quando l'endpoint vulnerabile consente anche operazioni di scrittura, l'impatto si estende all'integrità delle risorse: un attaccante può modificare dati appartenenti ad altri utenti, alterare lo stato di un ordine, cambiare le impostazioni di un account che non gli appartiene. Se l'operazione esposta è una cancellazione, la conseguenza diventa la perdita di disponibilità della risorsa per il legittimo proprietario.

Un aspetto che distingue BOLA da molte altre classi di vulnerabilità è la sua natura sistematicamente ripetibile: una volta individuato un endpoint privo del controllo appropriato, lo stesso schema di attacco può essere applicato a un numero elevato di identificatori, trasformando quella che sembra una singola falla puntuale in un canale di accesso a porzioni estese della base dati.

Il rischio assume connotati particolari negli ambienti multi-tenant e cloud, dove più clienti condividono la stessa infrastruttura applicativa e la separazione tra i rispettivi dati dipende interamente dalla correttezza della logica di autorizzazione, non da una separazione fisica delle risorse. In un simile contesto, una singola vulnerabilità BOLA non compromette soltanto i dati di un utente, ma può erodere il confine logico tra tenant distinti, con implicazioni che vanno oltre il singolo incidente e toccano la fiducia complessiva riposta nell'infrastruttura condivisa.

Mitigazione La prevenzione di BOLA richiede che ogni accesso a una risorsa identificata da un parametro fornito dal client sia accompagnato da una verifica esplicita del diritto dell'utente su quello specifico oggetto, eseguita interamente lato server. Nessuna informazione proveniente dal client, incluso l'identificatore stesso della risorsa, può essere considerata prova di autorizzazione: l'identificatore indica quale oggetto viene richiesto, non chi ha diritto di accedervi.

Un principio applicabile in modo trasversale è quello del *least privilege*, secondo cui un soggetto dovrebbe disporre soltanto dei permessi strettamente necessari a svolgere le proprie funzioni, senza margini di accesso più ampi per comodità implementativa. Applicato al livello oggetto, questo si traduce nella necessità di associare esplicitamente ogni utente, o ruolo, alle risorse su cui ha effettivamente

diritto di operare, evitando che l'accesso derivi da assunzioni implicite sulla struttura delle richieste.

Dal punto di vista architetturale, è preferibile centralizzare la logica di autorizzazione in un livello comune — un modulo, un middleware o un servizio dedicato — piuttosto che reimplementarla in modo indipendente in ciascun endpoint. Questa scelta non elimina la necessità di applicare il controllo in ogni punto di accesso, ma riduce la probabilità che errori o dimenticanze locali producano percorsi non protetti, e semplifica la verifica di coerenza tra le diverse funzioni esposte dall'API.

La coerenza dei controlli tra operazioni di lettura, scrittura e cancellazione sulla stessa risorsa merita infine attenzione specifica: la protezione di un'operazione non implica automaticamente la protezione delle altre, ed è in questa asimmetria che si annida buona parte delle istanze reali di BOLA.

Le sottosezioni seguenti trattano in forma più sintetica le restanti categorie della OWASP API Security Top 10, a partire da API2:2023.

2.2.2 API2:2023 – Broken Authentication

Definizione e collocazione Broken Authentication occupa la seconda posizione della classificazione OWASP e si colloca concettualmente a monte di BOLA: mentre quest'ultima riguarda il controllo sull'accesso a uno specifico dato o risorsa, Broken Authentication compromette la capacità stessa del sistema di stabilire con certezza l'identità di chi effettua la richiesta. Un fallimento in questa fase rende vano qualsiasi controllo successivo, poiché un meccanismo di autorizzazione perde ogni efficacia se l'identità su cui si fonda può essere falsificata o aggirata [6]. A differenza di altre categorie più circoscritte, questa voce raggruppa un insieme eterogeneo di debolezze nei processi di verifica dell'identità: dalla gestione poco attenta dei token di sessione all'assenza di vincoli sul numero di tentativi di accesso consentiti. Un esempio comune riguarda i messaggi di risposta del server: se il sistema indica esplicitamente quale dei due campi, nome utente o password, sia errato, fornisce involontariamente all'attaccante una conferma sull'esistenza di quell'account. Parallelamente, l'assenza di limitazioni sulla frequenza delle richieste lascia l'interfaccia di login esposta ad attacchi automatizzati volti a indovinare le credenziali su larga scala.

Cause e meccanismi tecnici tipici Spesso queste problematiche nascono dal tentativo di implementare soluzioni di autenticazione proprietarie o non standard. Come evidenziato dalla documentazione OWASP, strumenti come le API key o protocolli di autorizzazione come OAuth vengono talvolta impiegati impropriamente per identificare gli utenti in scenari per cui non sono stati progettati, creando falle strutturali [6]. Un ulteriore fattore critico è la mancanza di controlli sul volume dei tentativi di accesso. Endpoint di autenticazione e flussi di recupero password privi di rate limiting risultano vulnerabili ad attacchi in cui elenchi di credenziali già compromesse altrove vengono provati in massa, sfruttando la tendenza diffusa a riutilizzare le medesime password su più servizi [7]. Infine, la gestione dei token di sessione (come i JSON Web Token, o JWT) introduce ulteriori insidie: errori nella convalida della firma crittografica, chiavi di cifratura deboli, mancato controllo dei tempi di scadenza o assenza di meccanismi di revoca immediata permettono a un attaccante di forgiare token validi o di continuare a utilizzare credenziali che dovrebbero essere

già decadute [8].

Impatto L'impatto di Broken Authentication è per sua natura sistemico: assumendo il controllo dell'identità di un utente, l'attaccante ne eredita tutti i privilegi operativi senza che il backend possa distinguere le azioni legittime da quelle malevole [6]. Ciò può tradursi non solo nell'accesso a dati riservati, ma anche nella manomissione dei parametri di sicurezza dell'account (come la modifica dell'indirizzo di recupero), trasformando un'intrusione temporanea nella perdita definitiva del profilo.

Mitigazione La contromisura principale consiste nell'affidarsi a standard industriali consolidati e a librerie verificate, evitando soluzioni "fai-da-te" per la gestione di token e password [9]. A livello applicativo e infrastrutturale è indispensabile introdurre politiche di rate limiting dedicate e restrittive sugli endpoint di login, adottare l'autenticazione a più fattori (MFA) dove possibile e imporre una nuova conferma delle credenziali prima di eseguire modifiche critiche sul profilo utente [10].

Rilevabilità Sul piano della rilevabilità l'analisi automatica si divide in due direttrici, a seconda della natura del difetto.

1. **Analisi dei token e della crittografia.** Le debolezze che riguardano i JWT e gli altri formati di sessione dipendono da regole deterministiche e ben definite, come la mancata verifica della firma o l'uso di algoritmi insicuri. Gli strumenti riescono quindi a individuarle con buona affidabilità, sia analizzando staticamente il codice sorgente per intercettare configurazioni errate delle librerie crittografiche, sia generando dinamicamente token manipolati per verificare se il backend li accetti [11, 12].
2. **Flussi di autenticazione e abusi comportamentali.** L'assenza di rate limiting e l'esposizione a tentativi di accesso massivi richiedono invece un monitoraggio a runtime: più che da un'ispezione sintattica del codice, queste carenze emergono dall'analisi dinamica del traffico o dai registri di sistema, dove si manifestano come picchi anomali di richieste fallite concentrate in brevi finestre temporali [7].

2.2.3 API3:2023 – Broken Object Property Level Authorization

Definizione e collocazione La *Broken Object Property Level Authorization* (BOPLA) identifica una classe di vulnerabilità nella quale i controlli di autorizzazione applicati alle API non considerano adeguatamente le singole proprietà degli oggetti gestiti dall'applicazione. Il problema non riguarda quindi soltanto la possibilità per un utente di accedere a un determinato oggetto, ma anche quali attributi di quell'oggetto possa effettivamente leggere o modificare [4].

La distinzione rispetto alla *Broken Object Level Authorization* (BOLA) è rilevante. Nel caso di BOLA, il controllo di autorizzazione fallisce a livello dell'oggetto: un utente riesce, per esempio, ad accedere a una risorsa appartenente a un altro utente manipolando l'identificativo della risorsa. Nel caso di BOPLA, invece, l'utente può avere legittimamente accesso all'oggetto, ma ottenere o modificare proprietà che non rientrano nei suoi privilegi. L'oggetto, quindi, non costituisce necessariamente la risorsa a cui viene negato l'accesso; è una sua specifica proprietà a essere esposta o manipolabile senza un'adeguata autorizzazione.

Il rischio è particolarmente rilevante nelle API REST, dove è frequente che un endpoint restituisca direttamente la rappresentazione serializzata di un oggetto. Una struttura interna composta da numerosi attributi può così diventare, in assenza di una selezione esplicita, il corpo stesso della risposta inviata al client.

Cause e meccanismi tecnici tipici Una delle cause più comuni è la sovrapposizione tra il modello utilizzato internamente dall'applicazione e quello esposto attraverso l'API. Un oggetto memorizzato nel database può contenere, ad esempio, proprietà quali `id`, `name`, `email`, `role` e `internalStatus`. Non tutte queste proprietà hanno però la stessa finalità né devono necessariamente essere accessibili allo stesso insieme di utenti.

Un'implementazione che serializza automaticamente l'intero oggetto può produrre una risposta contenente anche attributi che il client non dovrebbe conoscere. In questo caso il problema non deriva necessariamente da un accesso non autorizzato all'oggetto, ma dal fatto che l'API non applica una politica distinta alle proprietà che lo compongono. L'interfaccia grafica potrebbe nascondere determinati campi, ma

ciò non costituisce una misura di sicurezza: il contenuto della risposta HTTP rimane osservabile dal client e può essere analizzato direttamente.

Un secondo meccanismo riguarda le operazioni di scrittura. Framework e librerie possono consentire il *binding* automatico dei parametri ricevuti dal client verso gli attributi di un oggetto applicativo. Se il server accetta un payload come:

```
{
  "name": "Mario",
  "email": "mario@example.com"
}
```

un attaccante può provare ad aggiungere proprietà che non erano previste dal normale flusso applicativo:

```
{
  "name": "Mario",
  "email": "mario@example.com",
  "role": "admin"
}
```

Se il campo `role` viene accettato e aggiornato senza un controllo specifico, l'utente può modificare una proprietà interna che dovrebbe essere gestita esclusivamente dal sistema. Lo stesso principio può riguardare attributi quali `isAdmin`, `approved`, `price` o `ownerId`, a seconda del modello utilizzato dall'applicazione.

Il problema può quindi manifestarsi in due direzioni differenti: lettura non autorizzata di proprietà e modifica non autorizzata di proprietà. Nel primo caso l'attaccante osserva informazioni che non dovrebbe ricevere; nel secondo riesce a influenzare direttamente lo stato dell'applicazione attraverso attributi che non dovrebbero essere controllabili dal client.

Impatto L'impatto di una vulnerabilità BOPLA dipende principalmente dal tipo di proprietà coinvolta e dalle funzioni che la utilizzano. La semplice esposizione di un attributo può comportare una violazione della riservatezza, mentre la possibilità di modificarlo può compromettere l'integrità dei dati o il funzionamento dell'applicazione.

Nel caso di proprietà contenenti informazioni personali, finanziarie o relative all'organizzazione interna del sistema, l'esposizione può determinare una divulgazione di dati a soggetti non autorizzati. La possibilità di modificare determinate proprietà può invece portare alla perdita o alla corruzione dei dati. In situazioni più critiche, una proprietà utilizzata nei meccanismi di autorizzazione o nella gestione dello stato dell'account può essere sfruttata per ottenere un'escalation dei privilegi o contribuire alla compromissione dell'account. La gravità della vulnerabilità dipende quindi dal significato che quella proprietà assume all'interno dell'applicazione, più che dal meccanismo che ne consente l'accesso.

Mitigazione La mitigazione di BOPLA richiede che l'autorizzazione venga applicata non soltanto all'endpoint o all'oggetto, ma anche alle proprietà che possono essere restituite o modificate. La prima misura consiste quindi nel definire esplicitamente quali attributi ogni endpoint può esporre e quali possono essere accettati in input.

Per le risposte è preferibile evitare i meccanismi di serializzazione automatica, che restituiscono l'oggetto così com'è rappresentato internamente, e selezionare invece le proprietà necessarie alla singola operazione.

Per le operazioni di scrittura è necessario limitare esplicitamente gli attributi modificabili dal client. Una tecnica comune consiste nell'utilizzare una *whitelist* delle proprietà ammesse, evitando che il payload venga associato automaticamente a tutti gli attributi dell'oggetto.

Rilevabilità La rilevazione di BOPLA si articola in due verifiche distinte, corrispondenti alle due direzioni in cui una proprietà può attraversare il confine fra client e server.

La prima riguarda le proprietà esposte in risposta a una richiesta legittima, e si riduce al confronto fra gli attributi effettivamente restituiti e quelli previsti per il ruolo o il contesto dell'utente. È la verifica più diretta, perché l'evidenza è contenuta nella risposta stessa.

La seconda riguarda le proprietà accettate in ingresso, ed è più onerosa perché il server non le dichiara. Individuarle richiede di modificare il payload delle ri-

chieste, eventualmente ricorrendo a tecniche di *fuzzing*⁵ per scoprire attributi non documentati, e di verificare poi se il server ne recepisca effettivamente il valore.

Le due verifiche non sono sostituibili l'una con l'altra: un'API può accettare in input una proprietà sensibile che non restituisce mai, oppure restituire dati riservati senza consentirne la modifica. Un'analisi che si limitasse a una sola direzione lascerebbe scoperta metà del problema.

2.2.4 API4:2023 – Unrestricted Resource Consumption

Ogni richiesta servita da un'API consuma risorse: banda, CPU, memoria, storage e, con frequenza crescente, servizi a pagamento di terze parti. API4:2023 raccoglie i casi in cui almeno uno dei limiti che dovrebbero governare quel consumo è assente o dimensionato male, fra cui i timeout di esecuzione, la memoria allocabile e il numero di processi [13]. Non è il difetto di una funzione specifica, ma una proprietà trasversale di qualunque endpoint: i limiti di consumo sono la difesa in profondità di altre categorie, e la loro assenza si intreccia con gli endpoint di autenticazione di API2, con l'automazione dei flussi di business di API6, con la copertura incompleta della superficie esposta di API9 e con le chiamate verso servizi esterni di API10.

La radice comune è un'asimmetria: produrre una richiesta costa al client quasi nulla, servirla può costare al server una quantità che cresce in funzione di parametri che il client controlla, e finché quella funzione di costo non ha un limite superiore l'API resta esposta. I meccanismi sono tre. Il volume puro, dove più insidiosa dell'assenza del limite è la sua presenza in una sola dimensione, perché un tetto per indirizzo IP cade contro un attacco distribuito e uno per endpoint non intercetta il batching. La moltiplicazione di operazioni dentro una richiesta sola, illustrata dallo scenario GraphQL in cui centinaia di mutazioni aggregate in un unico batch saturano la memoria aggirando sia il limite di frequenza sia quello sulla dimensione delle immagini [13, 14]. Infine il costo algoritmico della singola richiesta [15], con il ReDoS come istanza più diffusa, documentata nei server JavaScript dove il thread singolo rende una richiesta sufficiente a congelare il processo [16]. Vi si aggiunge

⁵*Fuzzing*: tecnica di collaudo che sottopone al sistema grandi quantità di input generati automaticamente, spesso malformati o inattesi, per osservarne le reazioni anomale.

la causa che l'edizione 2023 porta in primo piano, l'assenza di soglie di spesa verso i fornitori esterni, dove la risorsa non è computazionale ma economica. Che limiti tanto semplici manchino dipende da due fatti: possono risiedere nel framework, nel proxy, nel gateway⁶ o presso il fornitore cloud, e quando ogni livello presume che se ne occupi un altro non se ne occupa nessuno; e la loro assenza non produce errori, sicché un endpoint privo di limiti supera ogni test funzionale.

L'esito immediato è il denial of service per esaurimento, che CWE-770⁷ enumera come conseguenza principale sulla disponibilità [17], ma il punto di rottura è la logica applicativa anziché la banda: una sola richiesta ReDoS⁸ o un solo batch GraphQL bastano, e nessuna difesa perimetrale basata sul volume li riconosce come anomali.

Nella mitigazione nessun livello basta da solo: i limiti fisici su memoria, CPU e processi sono dichiarati alla piattaforma, mentre all'ingresso dei dati va fissata una dimensione massima per ogni parametro e payload, vincolo che il contratto OpenAPI rende verificabile da uno strumento. Il rate limiting, segnalato al client con il codice 429 [18, 19], va tarato per endpoint e affiancato da un limite sulle esecuzioni della singola operazione per utente, che il conteggio delle chiamate non cattura [13], e si irrigidisce sugli endpoint di autenticazione [6, 9]; NIST lo colloca nel gateway anziché nel singolo servizio [20], scelta corretta purché ogni host esposto stia dietro quel gateway. Per la famiglia algoritmica servono timeout e motori a tempo lineare [16], per GraphQL l'analisi statica del costo di una query prima di eseguirla [14], per la dimensione economica soglie di spesa o allarmi di fatturazione.

Sulla rilevabilità le due fonti primarie sembrano contraddirsi: OWASP la giudica facile, perché richieste con parametri di dimensione crescente e l'osservazione di stato, tempo e lunghezza della risposta bastano a far emergere il problema [13], mentre CWE-770 giudica di utilità limitata l'analisi statica per le allocazioni prive di

⁶*Gateway*: componente posto davanti ai servizi applicativi che riceve tutte le chiamate esterne e vi applica funzioni comuni quali autenticazione, instradamento e limiti di frequenza.

⁷CWE (*Common Weakness Enumeration*): il catalogo pubblico delle debolezze software mantenuto da MITRE; ogni voce, indicata dalla sigla CWE seguita da un numero, descrive una classe di difetto indipendentemente dal prodotto in cui compare.

⁸ReDoS (*Regular expression Denial of Service*): un input costruito ad arte che fa degenerare il tempo di valutazione di un'espressione regolare, saturando la CPU con una richiesta sola.

limite [17]. La contraddizione è apparente, perché OWASP descrive un tester in *black box*⁹ e CWE uno strumento che legge il codice. Sul versante dinamico l'evidenza è consolidata, dalle tecniche basate sulla generazione di molte richieste in un intervallo breve al rilevamento per iniezione di carico [17, 21], e il test differenziale osserva il sistema nel suo insieme, vedendo il limite ovunque sia applicato ma solo sull'host che interroga. Sul versante statico la letteratura si concentra su sottoclassi ben delimitate, il ReDoS e il costo delle query GraphQL, mentre manca un corpo di lavori sottoposti a revisione paritaria sul rilevamento dell'assenza di rate limiting, timeout o limiti di paginazione nel codice di API REST. Se ne ricava la natura del segnale: in una BOLA l'assenza del controllo nel gestore coincide con la vulnerabilità, mentre qui l'analisi del codice può affermare solo che a un dato livello il limite non c'è, non che il difetto esista, perché potrebbe risiedere nel gateway. È un'assenza locale, che diventa segnalazione fondata solo quando un secondo segnale la conferma, e lo scenario di API9 mostra il caso simmetrico, in cui il limite presente a un livello non copre tutti gli host esposti [22]: la correlazione fra segnali eterogenei non è dunque un raffinamento dell'analisi, ma la condizione perché un giudizio sia possibile.

2.2.5 API5:2023 – Broken Function Level Authorization

Un'API espone operazioni, e non tutte sono destinate a tutti. Creare un invito con privilegi amministrativi, esportare l'elenco degli utenti, cancellare un account altrui sono funzioni riservate a un ruolo, e il controllo che verifica quel ruolo prima di eseguirle è l'autorizzazione a livello di funzione: API5:2023 raccoglie i casi in cui tale controllo manca o è aggirabile, e un utente ordinario riesce a invocare funzioni riservate [23]. La differenza rispetto ad API1:2023 è di livello, ed è l'unica che OWASP fissi in modo prescrittivo: nella BOLA l'accesso all'endpoint è previsto e la violazione avviene manipolando l'identificatore dell'oggetto, mentre se l'attaccante raggiunge una funzione cui non dovrebbe accedere il caso ricade sotto la BFLA [5]. Il riferimento tassonomico è CWE-285 *Improper Authorization*, che MITRE sconsiglia però di usare

⁹*Black box*: modalità di analisi che osserva il sistema solo dall'esterno, attraverso le risposte alle richieste, senza accesso al codice sorgente; vi si contrappone l'analisi *white box*, che legge il codice.

per mappare vulnerabilità reali, rimandando alle voci figlie CWE-862 quando il controllo è assente e CWE-863 quando esiste ma decide male [24].

Il controllo funzionale manca perché non ha un luogo obbligato: OWASP osserva che l'autorizzazione su una funzione è gestita tramite configurazione oppure a livello di codice, e che scrivere controlli corretti è confuso, date le molte gerarchie di ruoli e gruppi delle applicazioni moderne [23]. Vi si somma il valore predefinito, perché una rotta aggiunta a un controller eredita il routing e l'autenticazione, non l'autorizzazione: nasce raggiungibile e tale resta. Sfruttarla non richiede manipolazioni, bastano chiamate legittime verso un endpoint cui non si dovrebbe accedere, e la struttura prevedibile delle API rende il tentativo facile, sostituendo GET con PUT oppure `users` con `admins`: la stessa regolarità che rende un'API documentabile la rende enumerabile. Lo scenario di riferimento è un'applicazione ad accesso su invito, in cui l'app mobile recupera un invito già emesso con una chiamata del tutto ordinaria:

```
GET /api/invites/{invite_guid} HTTP/1.1
Host: api.example.com
Authorization: Bearer <token_utente_normale>
```

La risposta espone, fra gli altri campi, il ruolo associato all'invito. L'attaccante ne cambia allora metodo ed endpoint, riutilizzando lo stesso token e senza manipolare alcun identificatore:

```
POST /api/invites/new HTTP/1.1
Host: api.example.com
Authorization: Bearer <token_utente_normale>
Content-Type: application/json

{"email": "attaccante@example.com", "role": "admin"}
```

L'operazione è riservata agli amministratori, ma soltanto attraverso una console priva di controllo di funzione: il server la esegue [23]. Il controllo, in un certo senso, esisteva, ma stava nella console, nel fatto che solo un amministratore vedesse quel

pulsante. Tre osservazioni spostano infine il difetto fuori dal singolo gestore: il percorso non dice se un endpoint sia amministrativo, perché le funzioni riservate stanno spesso sotto `api/users` anziché sotto `api/admins` [23]; i verbi HTTP lasciati abilitati per inerzia riaprono l'operazione da una via non presidiata [25]; le versioni obsolete continuano a servire il gestore precedente al controllo [22].

OWASP colloca l'impatto tecnico nella fascia grave, con esiti che vanno dalla divulgazione alla perdita o corruzione dei dati, fino all'interruzione del servizio [23]. Il danno non scala però con il numero di identificatori, bensì con la potenza della funzione raggiunta: una BOLA restituisce un record per richiesta, mentre una BFLA su un endpoint di esportazione ottiene l'intera base dati con una chiamata sola; e poiché le funzioni riservate sono in prevalenza funzioni di scrittura, il difetto che espone l'esportazione degli utenti espone, sull'endpoint accanto, la loro cancellazione.

La prescrizione centrale riguarda il valore predefinito: negare ogni accesso e richiedere concessioni esplicite ai ruoli per ogni funzione [23], con un vantaggio che sta in come si manifesta l'errore, perché se il default è il diniego una dimenticanza produce un 403 che il primo test funzionale rivela, mentre se il default è il permesso la stessa dimenticanza non produce alcun segnale. Segue la revisione degli endpoint alla luce della logica di business e della gerarchia dei gruppi, lavoro che non ammette scorciatoie rispetto alla conoscenza del dominio. La terza è l'unica con una forma sintattica: i controller amministrativi devono ereditare da un controller astratto che applichi il controllo in base a ruolo o gruppo, misura che non aggiunge verifiche ma le riposiziona, perché ometterle richiede l'atto deliberato di collocare un gestore fuori dalla gerarchia, discontinuità visibile in revisione e cercabile da uno strumento. Nessuna delle tre risolve però la copertura, perché il controllo protegge solo la superficie servita dal codice che lo contiene [22, 25].

OWASP giudica facile la rilevabilità, perché le API sono strutturate e il modo di raggiungere una funzione è prevedibile [23], ma il giudizio riguarda metà del problema: formulare la chiamata candidata è facile, stabilire se la risposta fosse dovuta non lo è, perché un 200 OK attesta che l'operazione è stata eseguita, non che fosse legittimo eseguirla. Il contributo della letteratura non è ispezionare il codice ma ricostruire la specifica mancante, ricavandola dalle pagine raggiungibili da ciascun ruolo [26], dalla coerenza con cui i controlli sono applicati dentro una partizione per

ruolo [27] o dal contesto di autorizzazione [28]. Un'alternativa rinuncia a inferire e scrive regole di sicurezza per le API REST, verificandole con fuzzing stateful [29], mentre un lavoro recente affronta lo stesso problema in modalità black box [30].

2.2.6 API6:2023 – Unrestricted Access to Sensitive Business Flows

Esistono attacchi che non violano alcuna regola. Qui ogni singola chiamata è legittima: l'utente ha diritto di compierla, i controlli di autorizzazione funzionano, il codice si comporta come progettato, e il danno nasce dal numero. OWASP definisce vulnerabile l'endpoint che espone un flusso di business sensibile senza limitarne adeguatamente l'accesso, portando come esempi l'acquisto di un prodotto, la creazione di un commento e la prenotazione di una fascia oraria [31]. Segue l'affermazione che definisce davvero la categoria: il rischio di un accesso eccessivo cambia da settore a settore, e la creazione di post tramite script può costituire spam per un social network ed essere incoraggiata da un altro [31]. Nessuna proprietà del codice, del contratto o della configurazione rende dunque un endpoint vulnerabile ad API6: lo stesso identico endpoint è sicuro in un contesto e vulnerabile in un altro, e a decidere è il modello di business che gli sta dietro.

Coerentemente, la causa che OWASP indica non è tecnica: manca una visione d'insieme dell'API che ne supporti i requisiti di business [31]. Il difetto nasce nel raccordo fra chi progetta l'endpoint e chi conosce il valore economico di ciò che l'endpoint muove, e dove quel raccordo manca nessuno è nella posizione di accorgersene. Lo sfruttamento richiede di comprendere il modello di business, individuare i flussi sensibili e automatizzarne l'accesso, e soltanto l'ultimo passo produce traffico osservabile: il primo scenario lo illustra con una console da gioco annunciata in quantità limitata, acquistata da un programma eseguito il giorno del lancio e distribuita su indirizzi IP e località diverse [31], dove la distribuzione geografica non è un dettaglio di colore ma la contromisura dell'attaccante contro il limite per indirizzo, e spiega perché il rate limiting nella sua forma più semplice non basti.

Sull'impatto il documento OWASP è internamente incoerente: la tabella dei punteggi classifica l'impatto tecnico come moderato, mentre la descrizione degli impatti afferma che un impatto tecnico in generale non è atteso e porta come esempi di danno

l'impossibilità per gli utenti legittimi di acquistare un prodotto e l'inflazione nell'economia interna di un videogioco [31]. È la seconda formulazione a corrispondere agli scenari descritti, che non producono né violazione di riservatezza né alterazione di dati né indisponibilità del servizio: ciò che cambia sta fuori dal sistema e si misura in scorte sottratte, ricavi persi o costi di acquisizione pagati per utenti inesistenti. Ne segue un effetto collaterale rilevante, perché un incidente che non produce indisponibilità né perdita di dati non attiva le procedure di risposta e viene percepito come problema commerciale anziché di sicurezza.

La mitigazione proposta ha una struttura a due livelli in cui l'ordine conta: prima si identificano, sul piano del business, i flussi che un uso eccessivo può danneggiare, e solo dopo si sceglie il meccanismo tecnico adeguato al rischio individuato [31]. Nessuna categoria precedente prescrive una fase preliminare simile, perché in nessuna altra la configurazione corretta dipende dall'azienda che la adotta. I meccanismi elencati, dal fingerprinting del dispositivo al captcha, dall'individuazione di comportamenti non umani come il passaggio dal carrello all'acquisto in meno di un secondo fino al blocco dei nodi di uscita Tor, sono presentati come modi per rallentare le minacce automatizzate, non per impedirle, e la logica che li accomuna è economica: nessuno produce una decisione binaria affidabile, tutti spostano il costo dell'attacco fino a superare il profitto atteso.

Il rilevamento automatico si scontra infine con l'assenza di un oracolo¹⁰: nessun artefatto analizzabile, che sia codice, contratto o configurazione, risponde alla domanda se un dato flusso in una data azienda vada protetto, e la mancanza di una mappatura CWE registra il fatto che non esiste una debolezza del software da nominare. Le tecniche note per i difetti di logica applicativa cercano deviazioni da un comportamento atteso [32, 33], mentre qui il flusso viene percorso correttamente e l'attacco sta nel ripeterlo. Restano le contromisure comportamentali a runtime indicate da OWASP [31], che riconoscono l'automazione durante l'attacco anziché la vulnerabilità prima che avvenga.

¹⁰*Oracolo*: in collaudo del software, il criterio che permette di stabilire se un dato comportamento osservato sia corretto o difettoso.

2.2.7 API7:2023 – Server Side Request Forgery

Il traffico non viaggia in una sola direzione: verso servizi interni, archivi remoti e risorse indicate da chi interroga il servizio partono richieste che l'applicazione stessa emette, ed è su queste che agisce l'attaccante in API7:2023.

La causa immediata è l'assenza o l'inadeguatezza della validazione dell'URI fornito dal client, e sono i modelli di sviluppo moderni a incoraggiare l'accesso a URI di provenienza esterna: il difetto nasce quindi da una funzionalità voluta, perché nessuno introduce per sbaglio un endpoint che scarica un'immagine da un indirizzo scelto dall'utente. I due scenari del documento mostrano le due forme dell'attacco. Nella forma cieca un social network consente di indicare l'immagine del profilo tramite URL, e l'attaccante vi scrive un indirizzo interno:

```
POST /api/profile/upload_picture HTTP/1.1
Host: api.example.com
Content-Type: application/json
```

```
{"picture_url": "http://localhost:8080"}
```

Misurando il tempo di risposta deduce se la porta sia aperta, e scandisce così la rete interna [34]: l'informazione non arriva dal corpo della risposta, ma dalla sua latenza. Nella forma con ritorno un prodotto di sicurezza invia una richiesta di prova al webhook che l'utente sta configurando e gliene mostra la risposta, e l'attaccante vi indica l'indirizzo link-local del servizio di metadati dell'ambiente cloud — `http://169.254.169.254/latest/meta-data/` nella convenzione più diffusa — ottenendone le credenziali [34]: il webhook rende comune il difetto, l'indirizzo a percorso noto lo rende grave. Gli esiti vanno dall'enumerazione dei servizi interni per scansione di porte alla divulgazione di informazioni, dall'aggiramento di firewall all'uso del server come proxy [34]. L'elemento che distingue l'SSRF è il cambio di perimetro: l'attaccante non ottiene un dato, ottiene una posizione, quella del server, e il valore dell'attacco dipende perciò da cosa sia raggiungibile dall'interno. I servizi di metadati cloud convertono poi l'SSRF da ricognizione a compromissione, perché rispondono a un indirizzo noto senza autenticazione ed espongono le credenziali del ruolo associato alla macchina [35].

Sul piano della rete OWASP raccomanda di isolare il meccanismo di recupero delle risorse, che di norma mira a risorse remote e non interne: è la difesa più robusta perché non dipende dalla correttezza della validazione, dato che se il componente che effettua le richieste in uscita non ha rotta verso la rete interna un URL malevolo non produce nulla. Sul piano applicativo sono prescritte liste di elementi ammessi su tre dimensioni — origini remote attese, schemi di URL e porte, tipi di media accettati — ciascuna delle quali blocca una famiglia di aggiramenti diversa, cui seguono la disabilitazione delle redirezioni, un parser di URL collaudato e il divieto di restituire al client le risposte grezze [34]: quest’ultima non impedisce l’attacco, ma ne degrada l’esito dalla forma con ritorno a quella cieca. Va infine registrata un’ammissione che nessun’altra categoria contiene: il rischio SSRF non può sempre essere eliminato del tutto [34], perché funzionalità e vulnerabilità coincidono.

Per la prima volta nel capitolo le due fonti primarie concordano, e a favore dell’analisi statica: OWASP giudica facile la rilevabilità, con la sola eccezione della forma cieca [34], e CWE-918 indica come metodo l’analisi statica automatica — la ricerca dei percorsi che collegano una sorgente di input a un *sink*¹¹ — dichiarandone alta l’efficacia [35]. Il motivo è che qui l’oracolo è interno al codice: per API4 e API5 lo strumento osserva l’assenza di un controllo senza poter sapere se fosse dovuto, mentre per API7 la domanda non è se qualcosa manchi, ma se due punti del programma siano connessi, proprietà verificabile senza conoscere le intenzioni del progettista. La letteratura conferma il quadro e ne segnala i limiti ricorrenti, dall’analisi di flusso validata manualmente [36] alla potatura dei percorsi non sfruttabili per contenere i falsi positivi [37], fino al fuzzing affiancato alla marcatura dinamica [38]. Se ne ricavano tre problemi che anche il modulo descritto nella parte sperimentale deve affrontare: l’inventario dei sink, la sovra-marcatura delle stringhe contaminate che non producono un URL controllabile, e la validazione intermedia, dove distinguere un controllo efficace da uno aggirabile richiede di ragionarne la semantica. Nella verifica dinamica, infine, la forma con ritorno si conferma puntando l’endpoint verso un indirizzo controllato, mentre quella cieca richiede un canale laterale, tipicamente

¹¹*Sorgente e sink*: nell’analisi del flusso dei dati, la *sorgente* è il punto in cui un valore controllato dall’esterno entra nel programma e il *sink* è l’operazione sensibile in cui quel valore viene infine usato.

una risoluzione DNS: nella pipeline adottata nel seguito la mancata conferma non declassa quindi la segnalazione statica.

2.2.8 API8:2023 – Security Misconfiguration

Un controllo che manca, un limite non impostato, un URL non validato: ciascuna delle categorie precedenti indica un punto preciso in cui il difetto risiede. La Security Misconfiguration non ha un punto: OWASP dichiara che la configurazione insicura può verificarsi a qualunque livello dello stack, dalla rete all'applicazione, e i suoi otto indicatori attraversano tutti quei livelli, dall'*hardening*¹² assente ai verbi HTTP lasciati abilitati, dal TLS¹³ mancante ai messaggi di errore che espongono tracce dello stack [25]. Hanno una sola cosa in comune: nessuno è un errore di logica applicativa, perché il codice fa quello che deve ed è l'ambiente in cui gira a produrre l'esposizione. Lo conferma la particolarità tassonomica: fra le sette voci CWE associate alla categoria compare CWE-16 *Configuration*, di cui MITRE dichiara proibita la mappatura, invitando ad analizzare piuttosto il comportamento che la configurazione ha determinato [39]. Se ne ricava il criterio: la configurazione non è una debolezza, è il meccanismo attraverso cui una debolezza viene introdotta, sicché una politica CORS permissiva non è insicura in astratto, lo diventa perché produce un difetto di controllo di accesso.

La causa strutturale è che la configurazione non risiede in un luogo unico né sotto una responsabilità unica: si distribuisce fra codice, file di orchestrazione, gateway, impostazioni cloud e valori predefiniti delle librerie, ciascuno con un ciclo di vita proprio, e un valore corretto al rilascio può cessare di esserlo dopo un aggiornamento che nessuno ha collegato alla sicurezza. Il primo scenario mostra il predefinito pericoloso, ed è riconoscibilmente il caso Log4Shell: un'utilità di logging supporta le ricerche JNDI abilitate per impostazione predefinita, e quando espande un'intestazione conte-

¹²*Hardening*: insieme delle operazioni con cui si riduce la superficie di attacco di un sistema: disattivazione di funzioni e servizi non necessari, rimozione delle credenziali predefinite, applicazione delle impostazioni consigliate.

¹³TLS (*Transport Layer Security*): il protocollo che cifra e autentica il canale di comunicazione su cui viaggia il traffico HTTP.

nente un riferimento JNDI a un server LDAP controllato dall'attaccante ne scarica ed esegue l'oggetto malevolo [25], senza che nella catena vi sia alcun difetto del codice applicativo. Il secondo mostra il meccanismo opposto, l'impostazione assente: un social network espone i messaggi privati tramite `/dm/user_updates.json` e, non inviando l'intestazione `Cache-Control`, lascia che le conversazioni finiscano nella cache del browser, e qui non c'è nulla da disabilitare, c'è una direttiva che nessuno ha inviato. Resta poi la causa più diffusa, che l'elenco mette al secondo posto: patch mancanti e sistemi non aggiornati [25], difetto di manutenzione più che di configurazione.

Le configurazioni insicure, scrive OWASP, non solo espongono dati sensibili degli utenti, ma anche dettagli del sistema che possono condurre alla compromissione completa del server [25]: è la seconda parte a giustificare l'impatto severo, e i due scenari ne mostrano gli estremi, conversazioni private da un lato ed esecuzione di codice arbitrario dall'altro. Ciò che distingue la Security Misconfiguration è però la natura abilitante del difetto, che raramente costituisce l'attacco e quasi sempre la condizione che ne rende praticabile un altro: una traccia di stack non compromette nulla di per sé, ma rivela versioni e percorsi interni e abbassa il costo della ricognizione, mentre una politica di uscita permissiva trasforma un SSRF cieco in uno con ritorno. Vi si aggiunge l'ampiezza, perché un difetto di configurazione, essendo trasversale, non riguarda un endpoint ma tutti quelli serviti da quel componente.

Sul piano del processo OWASP chiede un hardening ripetibile, una revisione delle configurazioni sull'intero stack e un processo automatizzato che ne valuti l'efficacia in tutti gli ambienti, la cui implicazione comune conviene rendere esplicita: la configurazione va trattata come artefatto versionato e verificabile, non come stato accumulato di una macchina. Sul piano dei controlli le misure riguardano il canale cifrato verso ogni componente, i verbi HTTP da restringere, la politica CORS e le intestazioni di sicurezza, i tipi di contenuto accettati e gli schemi dei payload di risposta, comprese quelle di errore [25]; due meritano una nota, perché il canale cifrato è prescritto anche per le API interne, il che chiude l'assunto per cui il traffico interno sarebbe fidato, e lo schema dichiarato per le risposte di errore non è documentazione, ma un vincolo che a runtime impedisce la fuga di informazioni diagnostiche.

OWASP classifica la rilevabilità come facile con una motivazione che, per la prima

volta nel capitolo, rinvia agli strumenti: ne esistono di automatici per individuare e sfruttare configurazioni errate [25], formulazione simmetrica perché gli stessi strumenti servono a chi difende e a chi attacca, dato che ciò che cercano è pubblicamente noto. La ragione per cui il rilevamento funziona si comprende per contrasto: per l'autorizzazione funzionale lo strumento non sapeva se un controllo fosse dovuto e per il consumo di risorse non sapeva se un limite fosse applicato altrove, mentre per la configurazione l'oracolo è esterno e pubblico — valori raccomandati, predefiniti noti come pericolosi, versioni vulnerabili — e verificare la conformità a un riferimento è un problema di confronto, non di inferenza. Restano tre limiti: la copertura, perché ciò che non è dichiarato in un file non è ispezionabile prima dell'esecuzione; la deriva, perché un'analisi statica attesta ciò che un file dichiara, non ciò che è attivo sul sistema; e il contesto, perché una politica CORS che ammette qualunque origine è un difetto per un'API a client autenticati e una scelta legittima per un'API pubblica di sola lettura, sicché la verifica sintattica stabilisce solo se un'impostazione si discosti da un riferimento la cui pertinenza dipende dall'endpoint.

2.2.9 API9:2023 – Improper Inventory Management

Correggere un difetto presuppone sapere che esiste il componente che lo ospita, e fin qui quel componente è sempre stato noto: un handler, un limite, una configurazione. API9:2023 riguarda invece ciò che l'organizzazione non sa di avere: il difetto non è nel codice ma nella visibilità, e OWASP lo definisce come scarsa catalogazione degli host e delle versioni di un'API, articolandolo in due punti ciechi [22]. Il primo è quello della documentazione, perché restano senza risposta esplicite le domande su quale ambiente ospiti un'API, chi debba avervi accesso di rete e quale versione sia in esecuzione, e non esistono né un piano di dismissione né un inventario aggiornato degli host. Il secondo è quello dei flussi di dati, e ricorre quando dati sensibili raggiungono terze parti senza motivazione di business, approvazione o visibilità su cosa venga condiviso [22]. Anche qui il riferimento tassonomico è una voce non mappabile, CWE-1059 [40]: come per la Security Misconfiguration, la categoria non nomina un comportamento del prodotto, e nel suo caso non è un'imprecisione della tassonomia ma una proprietà del difetto, perché l'assenza di un inventario non è

qualcosa che il software faccia.

La causa è un'asimmetria fra due costi: esporre una nuova istanza richiede pochi minuti, catalogarla e prevederne la dismissione richiede un processo che nessuna pipeline esegue da sola, e ogni ambiente di collaudo è però un'API completa e raggiungibile, mentre l'inventario che dovrebbe elencarlo è un documento aggiornato a mano. Il primo scenario ne mostra la conseguenza: un social network protegge il recupero della password dalla forza bruta, ma il meccanismo non è nel codice dell'API, bensì in un componente interposto fra il client e l'host ufficiale, e un ricercatore individua `beta.api.socialnetwork.owasp.org`, che esegue la stessa API senza esserne coperto, e reimposta la password di qualunque utente indovinando un token di sei cifre [22]: il controllo esisteva ed era corretto, ma non copriva tutti gli host che servivano quel codice. Il secondo riguarda l'altro punto cieco, con un'applicazione malevola che ottiene il consenso di 270.000 utenti ma, attraverso di essi, accede alle informazioni private di 50.000.000 di contatti: qui non c'è alcun host dimenticato, manca la conoscenza di dove i dati finiscano.

OWASP classifica l'impatto tecnico come moderato, ma il dettaglio decisivo è che versioni diverse della stessa API sono talvolta collegate alla medesima base di dati [22]: l'host dimenticato non è una copia isolata, è una seconda porta sugli stessi dati. Ne discende il tratto distintivo, perché API9 non introduce una vulnerabilità propria ma rende raggiungibili quelle altrui e ne impedisce la correzione, dato che una patch si applica a ciò che si sa di avere in esecuzione; il giudizio di impatto moderato è quindi fuorviante, perché la gravità effettiva è quella del difetto peggiore che l'host non inventariato continua a servire.

Le contromisure muovono tutte dallo stesso presupposto, che la visibilità vada prodotta e non presupposta. Le prime sono di inventario — gli host, con ambiente, accesso di rete e versione, e i servizi integrati, con ruolo e sensibilità dei dati — cui segue la documentazione dell'API, che deve coprire autenticazione, errori, redirezioni, limitazione di frequenza, politica CORS ed endpoint. La misura più incisiva è però la successiva: generare la documentazione in modo automatico, con standard aperti e dentro la pipeline di integrazione continua [22], differenza non di grado ma di natura, perché trasforma la documentazione da documento parallelo, che diverge appena qualcuno dimentica di aggiornarlo, in artefatto prodotto dalla compilazione.

Restano le prescrizioni sulle versioni: applicare le misure di sicurezza esterne a tutte quelle esposte e non alla sola produzione, che è ciò che il primo scenario mostra disatteso, e non usare dati di produzione fuori dalla produzione.

OWASP giudica media la rilevabilità e non indica un metodo, limitandosi a osservare che una documentazione obsoleta rende più difficile individuare e correggere le vulnerabilità [22]: l'osservazione descrive un difetto che ostacola il proprio rilevamento, perché l'artefatto che servirebbe a trovarlo è quello che manca. Ciò che si può fare per via automatica è un confronto fra rappresentazioni: le rotte implementate nel codice ma non dichiarate nel contratto sono candidate a essere API ombra, mentre quelle dichiarate ma non implementate segnalano documentazione obsoleta, e nessuna delle due è una vulnerabilità, ma entrambe misurano la divergenza fra il modello e la realtà. Il confronto ha però un limite intrinseco, perché opera fra due rappresentazioni della stessa applicazione e non dice nulla sugli host su cui essa è distribuita: il primo scenario è precisamente il caso in cui codice e contratto coincidono e il difetto sta nella topologia di rete, il che richiede ricognizione infrastrutturale e non analisi del software. Per API9 non esiste dunque un rilevamento nel senso in cui esiste per le altre categorie, cioè l'individuazione di un difetto dentro un artefatto: esiste la misura di una divergenza fra rappresentazioni indipendenti dello stesso sistema — contratto, codice, inventario, traffico — il cui valore non sta nel produrre un verdetto, ma nel restituire la visibilità che il difetto sottrae.

2.2.10 API10:2023 – Unsafe Consumption of APIs

Il perimetro di un servizio non coincide con il codice che lo compone: ogni dipendenza esterna è una superficie che non gli appartiene. API10:2023 rovescia perciò la prospettiva dell'attacco diretto, presupposta dalle nove categorie precedenti: l'attaccante non colpisce l'API bersaglio ma un servizio da cui essa dipende, e il danno arriva per un canale che l'applicazione considera fidato; le note di rilascio dichiarano di aver introdotto la categoria perché gli attaccanti hanno cominciato a compromettere i servizi integrati anziché le API del bersaglio [41]. La causa che OWASP individua è psicologica prima che tecnica: gli sviluppatori si fidano dei dati ricevuti da terze parti più di quanto si fidino dell'input dell'utente, e adottano

standard più deboli quanto a trasporto, autenticazione e validazione [42]. Le cinque condizioni di vulnerabilità sono canale non cifrato, dati non validati, redirezioni seguite ciecamente, risorse non limitate e timeout assenti.

La causa strutturale è un confine di fiducia collocato nel punto sbagliato. Un servizio di terze parti occupa una posizione ambigua: è esterno all'applicazione, ma scelto deliberatamente, raggiungibile su un canale che essa ha configurato e spesso offerto da un'azienda reputata affidabile, e la sua risposta attraversa così un confine che nessuno ha presidiato, perché nessuno lo ha riconosciuto come tale. Il primo scenario mostra l'iniezione differita: un'API si affida a un servizio esterno per arricchire gli indirizzi commerciali degli utenti e ne salva i dati in una base di dati SQL, e gli attaccanti vi memorizzano un payload di SQL injection associato a un'attività creata da loro, poi inducono l'API a prelevarla [42]; ciò che lo distingue da un'iniezione ordinaria è che l'attaccante non invia mai il payload all'API bersaglio, e sanificare l'input degli utenti non serve, perché quell'input è un innocuo nome di attività. Il secondo mostra la redirezione seguita ciecamente: un'API invia informazioni mediche a un fornitore su connessione cifrata, gli attaccanti lo compromettono ed esso risponde con un 308 Permanent Redirect, sicché l'API ripete la richiesta identica, dati sensibili compresi, verso il loro server, e la fuga avviene per un comportamento predefinito del client HTTP che nessuno ha valutato come decisione di sicurezza. Il terzo scenario è il più istruttivo, perché la terza parte non deve nemmeno essere compromessa: un attaccante crea un repository Git denominato `' ; dropdb; --`, e l'applicazione che vi si integra ne usa il nome per costruire una query SQL ritenendolo sicuro [42], avendo confuso l'autenticità del canale con l'affidabilità del contenuto.

L'impatto varia a seconda di ciò che l'API bersaglio fa con i dati prelevati — esposizione di informazioni sensibili, molti tipi di iniezione o denial of service [42] — ed è classificato come severo. Ciò che caratterizza la categoria è che il difetto non ne determina l'entità, la quale è funzione di dove il dato non validato finisce: un'informazione prelevata da un fornitore e memorizzata può essere riletta mesi dopo da un componente che la tratta come dato interno, perché proviene dalla base di dati dell'organizzazione. Le contromisure prescritte sono quattro: valutare la postura di sicurezza dei fornitori al momento della scelta, imporre un canale di trasporto sicuro,

validare e sanificare i dati ricevuti dalle API integrate, mantenere una lista delle posizioni verso cui esse possono reindirizzare [42]. La terza è il nucleo della categoria ed è formulata negli stessi termini riservati all'input dell'utente: la contromisura consiste nell'abolire la distinzione fra le due sorgenti. Due delle cinque condizioni — timeout e limiti sulle risposte — restano però prive di contromisura corrispondente; quando il contratto del fornitore è noto, la risposta può essere validata rispetto a uno schema, che non intercetta un payload conforme ma trasforma la fiducia implicita in un vincolo dichiarato.

OWASP classifica infine la rilevabilità come media senza indicare come procedere [42]. Sul piano statico il problema è di configurazione più che di tecnica: l'analisi della contaminazione richiede un insieme di sorgenti e uno di destinazioni, e per convenzione consolidata le sorgenti sono i punti in cui entra l'input dell'utente, mentre per API10 la sorgente è il valore restituito da una chiamata HTTP in uscita, che un motore configurato secondo quella convenzione non considera contaminato, sicché un flusso dalla risposta di un fornitore a una query SQL non produce segnalazione benché sia identico a uno che partisse da un parametro di richiesta. Due dei cinque difetti si prestano invece a un rilevamento diretto, perché basta osservare la configurazione locale della chiamata: la redirectione seguita per impostazione predefinita e il timeout mancante, dove il segnale non è la presenza di un'opzione errata ma l'assenza di un'opzione che disattivi il predefinito insicuro. Sul piano dinamico la verifica richiede che il fornitore restituisca dati malevoli, il che è possibile solo se il servizio consente di immettervi contenuti oppure sostituendolo con un doppio in collaudo, che però verifica l'applicazione e non l'integrazione reale.

3.1 Approccio metodologico

3.1.1 Obiettivi e domande di ricerca

La rassegna del capitolo precedente restituisce un quadro disomogeneo: alcune categorie dell'OWASP API Security Top 10 dispongono di tecniche di rilevazione consolidate, altre sono trattate quasi solo in chiave dinamica, altre ancora non hanno una formulazione operativa che ne consenta il riconoscimento automatico. Il divario dipende da come si manifesta la violazione che definisce ciascuna categoria. Vi sono violazioni che lasciano una traccia strutturale negli artefatti di progetto (quali codice applicativo, definizioni infrastrutturali e specifica dell'interfaccia) e violazioni il cui riconoscimento presuppone di sapere quale uso dell'applicazione sia legittimo, informazione che nessuno di essi contiene.

Questa differenza corrisponde alla distinzione fra *vulnerabilità tecniche* e *vulnerabilità di logica di business* (*logic vulnerabilities*) [32, 33]. Le prime violano proprietà strutturali verificabili sugli artefatti di progetto; le seconde violano l'intento per cui il flusso applicativo è stato concepito. La stessa OWASP, nel motivare la categoria API6:2023, riconduce a questo carattere la resistenza dei difetti di logica di business

al rilevamento automatico: la correttezza è definita rispetto all'intento del dominio applicativo, non rispetto a una proprietà del codice [31].

Il presente lavoro applica sistematicamente tale distinzione alle dieci categorie dell'OWASP API Security Top 10 (2023) e ne verifica le conseguenze implementative con un'analisi multi-layer dei tre livelli indicati, integrata da evidenza dinamica dove la sola ispezione statica non basta. Su questa base si formula la domanda di ricerca principale.

Q RQ₁: *Quali categorie dell'OWASP API Security Top 10 (2023) sono riconducibili a vulnerabilità tecniche rilevabili mediante analisi statica multi-layer su codice applicativo, definizioni infrastrutturali e specifica dell'interfaccia, quali richiedono in aggiunta evidenza esterna agli artefatti, e di quale natura sia tale evidenza?*

La domanda si scompone in due interrogativi subordinati.

Q RQ_{1.1}: *Per le categorie riconducibili a vulnerabilità tecniche, quale combinazione di livelli di analisi è necessaria alla loro copertura, e quali condizioni di rischio emergono soltanto dalla congiunzione di segnali che, presi isolatamente, non costituiscono una violazione?*

Q RQ_{1.2}: *Per le categorie la cui proprietà violata non è interamente esprimibile sugli artefatti, quale componente resta riconoscibile staticamente e di quale natura è il residuo: accertabile sollecitando il sistema in esecuzione, colmabile estendendo il perimetro degli artefatti esaminati, esplicitabile come parametro dichiarato dell'analisi, oppure irriducibile alla conoscenza del dominio applicativo?*

3.1.2 Impostazione dello studio

Le domande non si risolvono sulla carta. Un criterio applicato con coerenza può dire quali categorie *sembrano* riconducibili a violazioni tecniche; se la riconduzione regga davvero lo si scopre solo costruendo i moduli di rilevazione e osservandoli all'opera su codice reale. Lo studio assume perciò una forma costruttiva.

Questo chiarisce il ruolo del sistema descritto nel seguito: è il banco di prova della classificazione, non l'oggetto della ricerca. Ogni categoria ricondotta a una violazione strutturale deve avere una regola che la riconosca; ogni categoria esclusa deve restare, coerentemente, senza modulo. Vale come evidenza soltanto ciò che è stato verificato: la classificazione secondo il criterio dichiarato, l'esito dei moduli su casi di prova e su un bersaglio realistico, e la motivazione documentata delle categorie non coperte. Una regola presente nel codice ma mai esercitata su un caso concreto non conta come copertura. Il capitolo formula il criterio, ne deriva i requisiti per lo strumento e si chiude con il disegno della valutazione; l'applicazione del criterio alle dieci categorie è presentata insieme ai risultati.

3.1.3 Il criterio di classificazione

Il criterio rende operativa la distinzione della Sottosezione 3.1.1. Per ogni categoria si individua la proprietà di sicurezza la cui violazione la definisce, attenendosi alla formulazione ufficiale OWASP [4], alle debolezze CWE associate e alla caratterizzazione emersa dalla rassegna.

Una categoria è detta *tecnica* quando esiste un predicato sugli artefatti di progetto che sia al tempo stesso necessario e sufficiente a stabilire la violazione, una volta fissati i parametri dichiarati dell'analisi. È detta *ibrida* quando esiste un predicato necessario ma non sufficiente: l'analisi statica riconosce una condizione senza la quale la violazione non può sussistere, ma la cui presenza non basta a stabilirla. È detta infine *di logica di business* quando non esiste alcun predicato necessario esprimibile sugli artefatti, poiché la proprietà violata è definita per riferimento all'intento applicativo.

È necessario delimitare quali parametri sia lecito fornire all'analisi: si ammettono unicamente configurazioni generali valide per l'intero sistema (come l'uso di canali cifrati o limiti di traffico attesi). Si escludono invece regole puntuali sui permessi degli utenti rispetto alle singole risorse, per evitare di inserire a priori la logica di autorizzazione che l'analisi stessa dovrebbe verificare.

Il residuo delle categorie ibride non è omogeneo, e distinguerne la natura è l'oggetto di RQ_{1.2}. Un residuo è *dinamico* quando la proprietà mancante si manifesta nella risposta del sistema a una sollecitazione mirata, tipicamente l'invio della stessa

richiesta con identità distinte. È *di perimetro* quando è colmabile estendendo l'analisi a un livello non ancora esaminato: il limite riguarda l'applicazione dell'analisi statica a un solo livello, non l'approccio in sé. È *di dichiarazione* quando richiede che qualcuno espliciti la configurazione attesa o il contenuto di un accordo esterno. È *irriducibile* quando nessuna osservazione del sistema, statica o dinamica, può restituirlo.

Il criterio presuppone infine che il sistema da valutare sia noto. La condizione non vale per API9:2023, che descrive l'esistenza di sistemi che l'organizzazione ignora di possedere: la categoria resta perciò fuori dalla tripartizione.

3.1.4 Requisiti derivati per lo strumento di rilevazione

La classificazione non descrive uno strumento, ma vincola ciò che uno strumento adeguato deve saper fare. I requisiti che seguono sono formulati come capacità richieste, indipendentemente dalle tecnologie; la loro traduzione in un'architettura concreta è mostrata nella Sezione 3.2 per la IaC misconfiguration e nei capitoli seguenti per le altre categorie.

Il primo requisito è l'analisi su più livelli. Le proprietà individuate dal criterio non risiedono tutte nello stesso luogo: alcune si manifestano nel codice applicativo, altre nelle definizioni infrastrutturali, altre nella specifica dell'interfaccia, e nessun livello è da solo sufficiente. Uno strumento che esamini un solo livello non può quindi coprire il perimetro, e ciascun livello richiede una rappresentazione e una tecnica di ispezione proprie.

Il secondo è la correlazione fra i livelli. Alcune condizioni non sono osservabili in alcun singolo livello, ma nella discrepanza fra due di essi: un percorso presente nel codice e assente dal contratto non è una violazione in nessuno dei due luoghi, eppure la divergenza segnala una superficie esposta e non governata; l'assenza di un limite nel codice cambia significato a seconda che le definizioni infrastrutturali ne prevedano uno altrove. Lo strumento deve quindi mettere in relazione segnali di livelli diversi e derivarne giudizi che nessun livello esprime da solo. Questa capacità risponde a RQ_{1.1}.

Il terzo è la verifica dinamica: per le categorie che richiedono un riscontro a runtime, lo strumento deve poter inviare richieste reali all'API e analizzarne le

risposte. L'analisi del codice rileva infatti solo un rischio potenziale, che rimarrebbe incerto senza una conferma attiva.

Il quarto è la configurabilità dei parametri: i valori e le soglie utilizzati dall'analisi devono essere espliciti e modificabili dall'utente, per evitare che impostazioni predefinite e cablate nelle regole impongano scelte arbitrarie come se fossero vincoli universali.

Il quinto è la tracciabilità: ogni segnalazione deve essere riconducibile alla categoria di cui costituisce evidenza e al livello di analisi che l'ha generata, perché una segnalazione priva di tale riconduzione può indicare un difetto reale ma non contribuisce a rispondere alle domande di ricerca.

3.1.5 Disegno della valutazione

La valutazione ha due scopi distinti: controllare che ogni modulo riconosca la condizione per cui è stato scritto, e osservare il comportamento dello strumento nel suo insieme su codice che non è stato scritto per essere analizzato.

Al primo scopo servono coppie di casi di prova, ciascuna formata da una versione che contiene la condizione da rilevare e da una che non la contiene: il modulo deve segnalare la prima e non la seconda. Le coppie coprono tutti e tre i livelli di analisi, così che nessun livello sia esercitato solo indirettamente attraverso il bersaglio realistico. Poiché casi di prova e regole hanno lo stesso autore, questa prova stabilisce che l'implementazione fa ciò che si intendeva, non che funzioni su codice reale.

Al secondo scopo serve OWASP crAPI [43], applicazione volutamente vulnerabile mantenuta dalla OWASP Foundation e quindi fissata a una revisione precisa per rendere i risultati riproducibili. Non essendo scritta dall'autore, riduce la circolarità della prima prova; resta però un bersaglio didattico, costruito perché le sue vulnerabilità siano riconoscibili, e i risultati non si possono estendere senza cautela ad applicazioni reali.

Per i casi di prova il riferimento di verità è noto per costruzione; per crAPI è stato ricostruito a mano, individuando nel codice i punti in cui si manifestano le vulnerabilità documentate dal progetto, ed è riportato per esteso perché i risultati vadano letti rispetto a ciò che si è assunto come vero. Una segnalazione conta co-

me corrispondente a una voce del riferimento quando riguarda la stessa categoria OWASP e lo stesso punto di esposizione dell'interfaccia, identificato da metodo e percorso normalizzato, qualunque sia il livello di analisi che l'ha prodotta; questa convenzione è stata fissata prima di osservare i risultati. Su questa base si registra, per ogni categoria, se i moduli segnalano i casi che contengono la condizione e non quelli che ne sono privi, e si distinguono le categorie senza segnalazioni in cui ciò discende dalla classificazione da quelle in cui indica una copertura mancante.

3.2 Rilevamento delle IaC misconfiguration

Questa sezione applica il metodo alla prima delle classi in esame, la *IaC misconfiguration* (Sottosezione 2.1.1): le configurazioni difettose dichiarate in file ispezionabili prima del rilascio. Qui tali file sono la definizione dell'infrastruttura come codice (i manifest Terraform, affiancati da Dockerfile e manifest Kubernetes gestiti dal medesimo strumento) e l'analisi è interamente statica: nessuna risorsa viene creata né interrogata. L'esposizione segue il percorso di una segnalazione: l'architettura entro cui lo scanner opera, lo strumento adottato e le policy pertinenti, l'adapter che ne traduce l'uscita nel modello di dominio, l'aggregazione con il calcolo del punteggio di rischio e, infine, la validazione condotta secondo il disegno della Sottosezione 3.1.5 con i limiti che ne emergono.

3.2.1 Architettura del sistema

Il sistema è organizzato secondo i principi della *Clean Architecture*: un livello di dominio che definisce le entità e i contratti, un livello applicativo che orchestra l'analisi, un livello infrastrutturale che incapsula gli strumenti esterni e un livello di presentazione che espone i risultati. La Figura 3.1 mostra i componenti raggruppati per livello e le dipendenze fra i livelli: la presentazione dipende dall'applicazione, l'applicazione dal dominio, e l'infrastruttura realizza le interfacce che il dominio dichiara. Il livello di presentazione offre due vie d'accesso alla stessa pipeline: un'interfaccia a riga di comando, che registra gli adapter, esegue l'orchestratore e scrive i

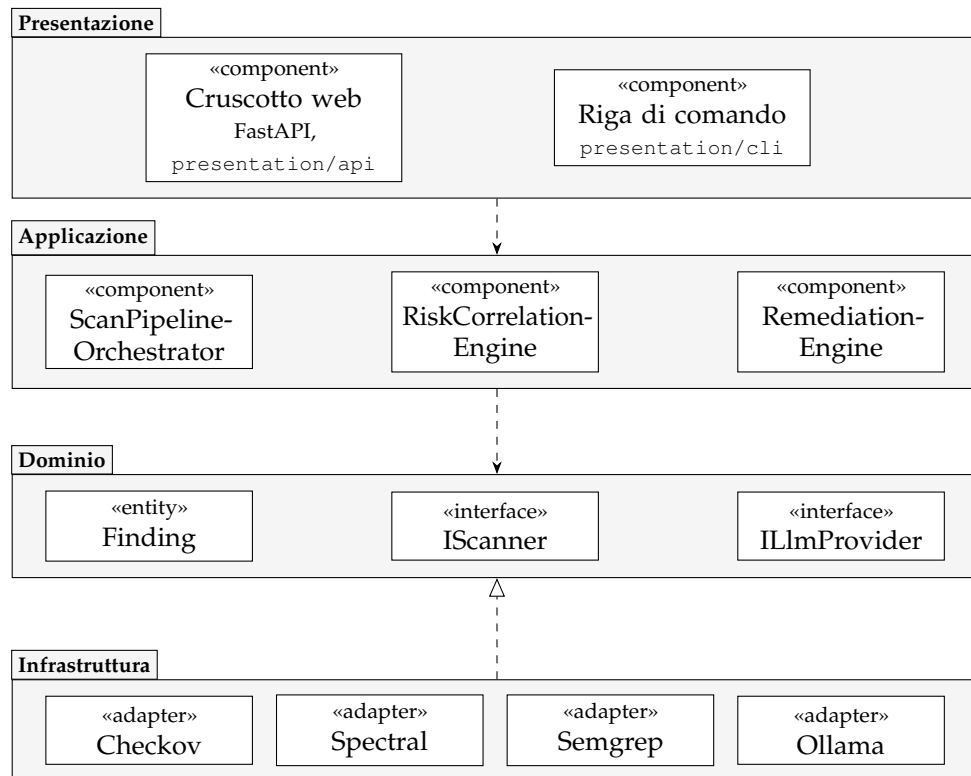


Figura 3.1: Diagramma UML dei componenti del sistema, raggruppati per livello architetturale. Le frecce tratteggiate indicano dipendenze; quella con la punta triangolare vuota indica la realizzazione delle interfacce di dominio da parte degli adapter.

rapporti su file, e un cruscotto web basato su FastAPI, che espone la stessa esecuzione come servizio e presenta i risultati in pagine interattive.

La separazione risponde direttamente alla tracciabilità richiesta nella Sottosezione 3.1.4: qualunque sia lo strumento che l’ha prodotta, ogni anomalia viene ricondotta a un’unica entità di dominio, il *Finding*, che la Figura 3.2 mostra insieme ai suoi oggetti valore, alle enumerazioni di severità e natura, all’interfaccia *IScanner* con l’adapter che la realizza e al motore di correlazione.

Un *Finding* risponde a tre domande. Da dove viene: la sorgente, la regola che l’ha generato e i dati grezzi dello strumento. Dove si trova: la posizione nel file, la risorsa infrastrutturale interessata e una chiave di correlazione. Quanto pesa: la categoria, la severità su cinque gradi (CRITICAL, HIGH, MEDIUM, LOW, più INFO per le note informative) e un grado di confidenza fra 0 e 1. A queste si aggiunge la *natura* del rischio, che distingue un’*esposizione*, quando la configurazione apre un varco sfruttabile direttamente, da un *irrobustimento*, quando manca una linea di difesa in profondità che da sola non costituisce un accesso. Quando la regola non permette

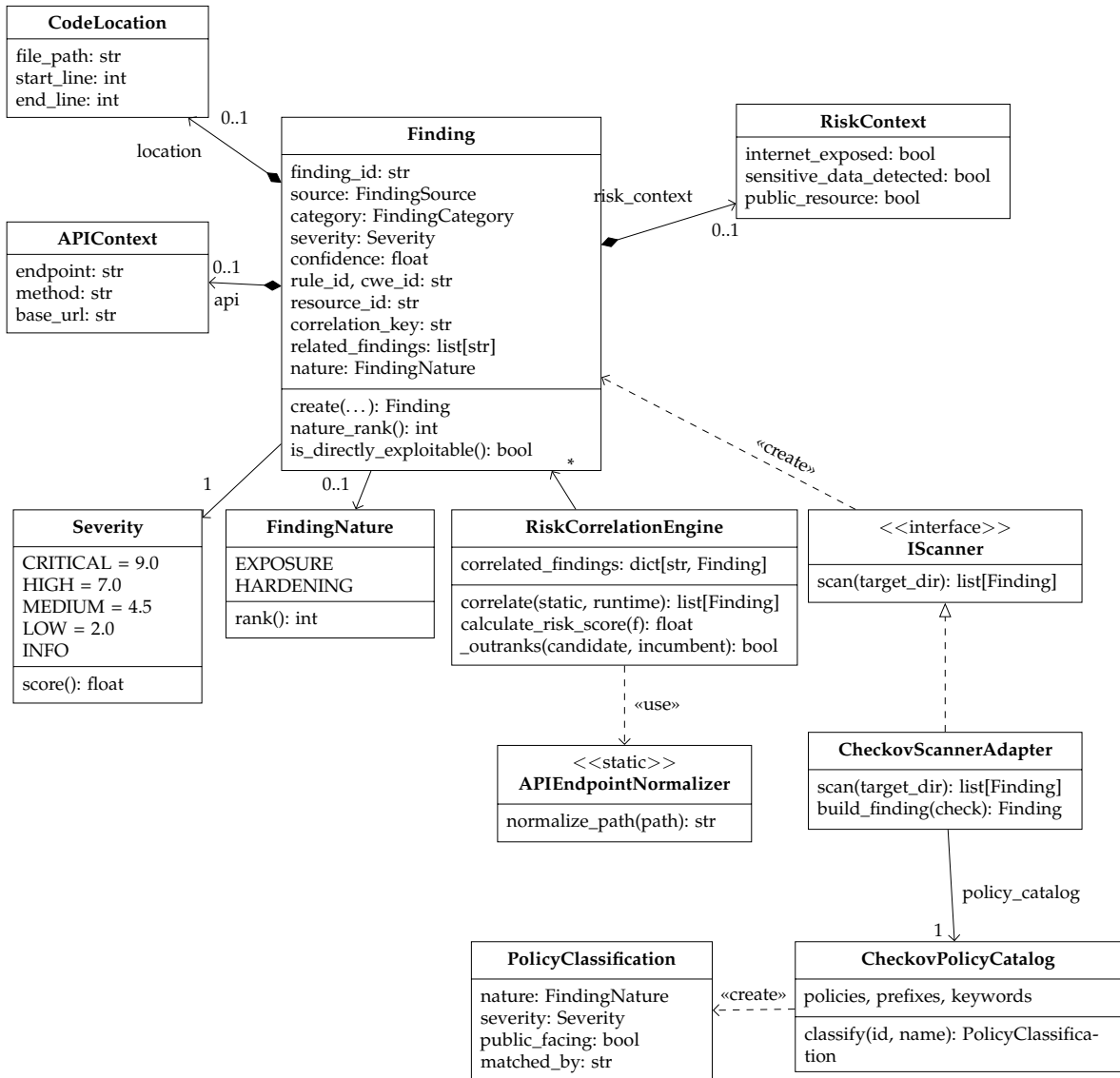


Figura 3.2: Diagramma UML delle classi del dominio e del motore di correlazione. *Severity* e *FindingNature* sono enumerazioni; *APIEndpointNormalizer* espone solo metodi di classe. Gli adapter di Semgrep e Spectral, non trattati, realizzano la stessa interfaccia.

di stabilire a priori da quale parte stia la segnalazione, la natura resta indefinita (un valore nullo nel dominio) e la segnalazione non classificata.

L'interfaccia astratta *IScanner* formalizza il contratto a cui ogni componente di analisi deve conformarsi: un metodo *scan* che riceve il percorso del bersaglio e restituisce una lista di *Finding*. È implementata dagli *adapter* che avvolgono strumenti esterni e hanno il compito di eseguire lo strumento, leggerne l'uscita e tradurla nel modello di dominio. L'adapter di cui si occupa questa sezione è quello di Chec-

kov [44], che analizza le definizioni infrastrutturali; la stessa interfaccia ha altre due realizzazioni, Semgrep per il codice applicativo e Spectral per la specifica dell'interfaccia, che servono altre categorie e restano fuori da questa sezione. L'orchestratore conosce gli scanner solo attraverso questa interfaccia, e l'esecuzione procede in due fasi: gli scanner registrati vengono eseguiti sul repository bersaglio, poi le segnalazioni raccolte vengono consegnate al motore di correlazione, che le aggrega per chiave e assegna a ciascuna un punteggio di rischio. Aggiungere un controllo significa registrare un nuovo adapter, senza modificare l'orchestratore.

3.2.2 Lo scanner Checkov e la traduzione nel modello di dominio

Checkov [44] è un analizzatore statico per Infrastructure-as-Code che legge i file di definizione (tra cui Terraform [45], CloudFormation, Kubernetes, Dockerfile, Helm e specifiche OpenAPI), ne costruisce un grafo di risorse e vi valuta un catalogo di policy. Le policy sono di due tipi: controlli sugli attributi di una singola risorsa, scritti in Python e identificati da `CKV_provider_n`, e controlli sulle relazioni fra risorse, espressi in YAML sul grafo e identificati da `CKV2_provider_n`; a questi si aggiunge un riconoscitore di segreti che esamina qualunque file e produce identificativi `CKV_SECRET_n`. Ogni valutazione di una policy su una risorsa dà un esito superato, fallito o non determinabile, con il file e l'intervallo di righe della risorsa.

Per lo scenario in perimetro, riguardante i permessi di condivisione aperti su risorse di storage, il catalogo contiene controlli che coprono le tre vie attraverso cui un bucket S3 può risultare pubblico: la lista di controllo d'accesso, la policy del bucket e il blocco dell'accesso pubblico a livello di account o di bucket. La Tabella 3.1 li elenca insieme ai controlli che Checkov valuta sulle stesse risorse senza che riguardino l'esposizione (registrazione degli accessi, versioning, cifratura con chiave gestita, replica, ciclo di vita, notifiche) e che lo strumento colloca sullo stesso piano dei primi. La distinzione fra esposizione e irrobustimento, essenziale per questo lavoro, nel catalogo di Checkov non esiste: è il sistema a introdurla attraverso un catalogo proprio delle policy, descritto nel seguito.

L'adapter esegue Checkov sulla sola directory bersaglio richiedendo l'uscita in formato JSON, e traduce ciascun controllo fallito in un Finding (Figura 3.3). La

Policy	Risorsa Terraform	Condizione verificata
<i>Esposizione dello storage</i>		
CKV_AWS_20	aws_s3_bucket	ACL che concede lettura pubblica
CKV_AWS_53–56	aws_s3_bucket_public_access_block	i quattro flag del blocco dell'accesso pubblico non attivi
CKV_AWS_70	aws_s3_bucket_policy	policy che concede un'azione a qualunque principal
CKV2_AWS_6	aws_s3_bucket	bucket privo di un blocco dell'accesso pubblico attivo (controllo sul grafo)
CKV2_AWS_65	aws_s3_bucket_ownership_controls	liste di controllo d'accesso non disabilitate
<i>Irrobustimento dello storage, senza esposizione</i>		
CKV_AWS_18	aws_s3_bucket	registrazione degli accessi assente
CKV_AWS_21	aws_s3_bucket	versioning non abilitato
CKV_AWS_145	aws_s3_bucket	cifratura non basata su chiave KMS
CKV_AWS_144	aws_s3_bucket	replica fra regioni assente
CKV2_AWS_61, 62	aws_s3_bucket	ciclo di vita e notifiche di evento assenti
<i>Altre risorse del bersaglio di prova</i>		
CKV_AWS_59	aws_api_gateway_method	metodo esposto senza autorizzazione
CKV_AWS_62, 63	aws_iam_role_policy	policy con privilegi amministrativi o azioni *
CKV_AWS_45	aws_lambda_function	segreto cablato nelle variabili d'ambiente
CKV_AWS_169	aws_sns_topic_policy	policy del topic aperta a qualunque principal

Tabella 3.1: Policy di Checkov 3.2.459 pertinenti allo scenario in perimetro e alle altre risorse del bersaglio di prova. Le denominazioni sono tradotte dai nomi ufficiali dei controlli.

categoria è derivata tramite parole chiave da identificativo, nome ufficiale e tipo di risorsa. Natura, severità ed esposizione pubblica provengono invece dal *catalogo delle policy*: un file YAML esterno che costituisce parametro dell'analisi e non costante del codice, in accordo con la configurabilità richiesta nella Sottosezione 3.1.4. Il catalogo risolve ciascun controllo secondo la priorità riassunta nella Tabella 3.2 (mappatura esplicita su 99 controlli, prefissi o parole chiave, con ripiego predefinito su segnalazione non classificata MEDIUM).

La confidenza del Finding non misura il controllo, che è deterministico sul file, ma la precisione con cui il catalogo lo ha interpretato: 1 per la mappatura esplicita, 0,9 per il prefisso, 0,8 per le parole chiave, 0,6 per il ripiego predefinito. Il Finding include titolo, intervallo di righe e record grezzo. Se il controllo è contrassegnato come pubblico, l'adapter vi associa un contesto di risorsa esposta a Internet e pubblica; se

riguarda segreti o cifratura a riposo, un contesto di dati sensibili. Come chiave di correlazione adotta l'ultimo segmento del nome della risorsa (ad esempio `bucket_1_public_acl` per `aws_s3_bucket.bucket_1_public_acl`), consentendo di raggruppare i rilievi riferiti alla medesima entità logica definita dallo sviluppatore.

Regola	Come risolve il controllo	Esempio
Mappatura esplicita	identificativo presente nel catalogo, con natura, severità ed eventuali flag di esposizione pubblica e di dati sensibili	CKV_AWS_20: esposizione, CRITICAL, pubblica
Prefisso	famiglia di controlli riconosciuta dall'inizio dell'identificativo	CKV_SECRET_*: esposizione, HIGH, dati sensibili
Parole chiave	termini cercati nel nome ufficiale del controllo; i gruppi difensivi precedono quelli di esposizione	<i>logging, versioning</i> : irrobustimento, LOW
Predefinito	nessuna regola soddisfatta	non classificato, MEDIUM

Tabella 3.2: Ordine con cui il catalogo delle policy assegna natura, severità e contesto a un controllo fallito di Checkov; la regola che risolve determina anche la confidenza. Le esposizioni hanno severità predefinita HIGH, gli irrobustimenti LOW, salvo indicazione esplicita.

3.2.3 Correlazione e punteggio di rischio

Il motore di correlazione indicizza le segnalazioni per chiave (nel caso delle segnalazioni infrastrutturali, il nome della risorsa) e fonde quelle che la condividono in un'unica voce. La rappresentante è scelta per natura e poi per severità: un'esposizione precede una segnalazione non classificata, che precede un irrobustimento; a parità di natura prevale la severità maggiore. Un controllo di natura inferiore non altera mai la severità della rappresentante e resta come sotto-elemento della voce, con la propria natura e severità: nulla viene scartato, e una sola esecuzione restituisce l'intero insieme delle condizioni da correggere. La Figura 3.4 riporta la sequenza della pipeline statica.

Il punteggio di rischio è un modello pesato proprio. Il CVSS sarebbe stato la scelta naturale, ma richiede un vettore che nessuno degli strumenti integrati produce:

$$R = \min(10, 0,6 S + 0,2 \cdot 10 C + 0,2 X), \quad (3.2.1)$$

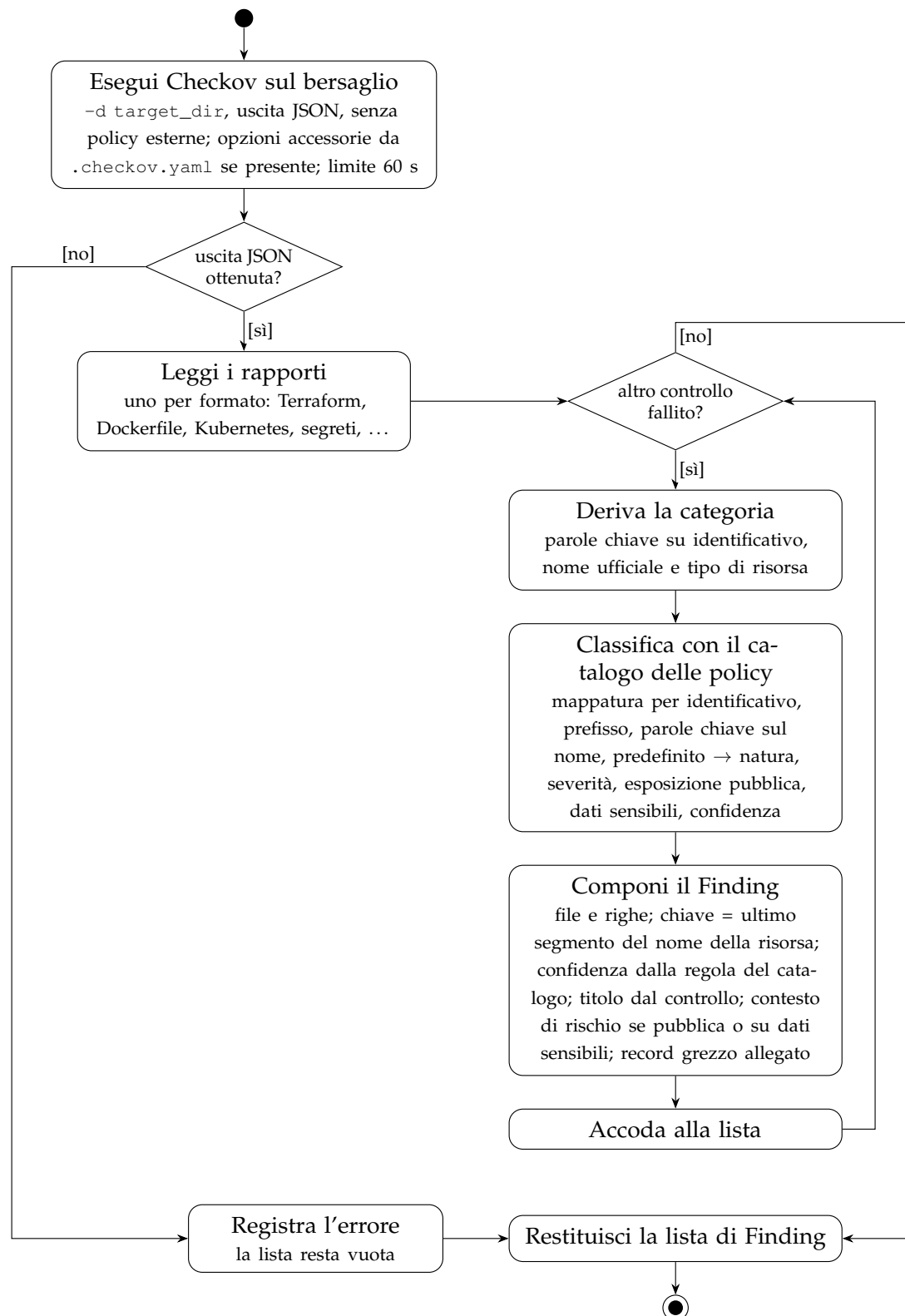


Figura 3.3: Diagramma UML di attività del metodo `scan` dell'adapter di Checkov. Ogni controllo fallito diventa un Finding: la categoria è derivata da identificativo, nome e tipo di risorsa; natura, severità ed esposizione pubblica provengono dal catalogo delle policy.

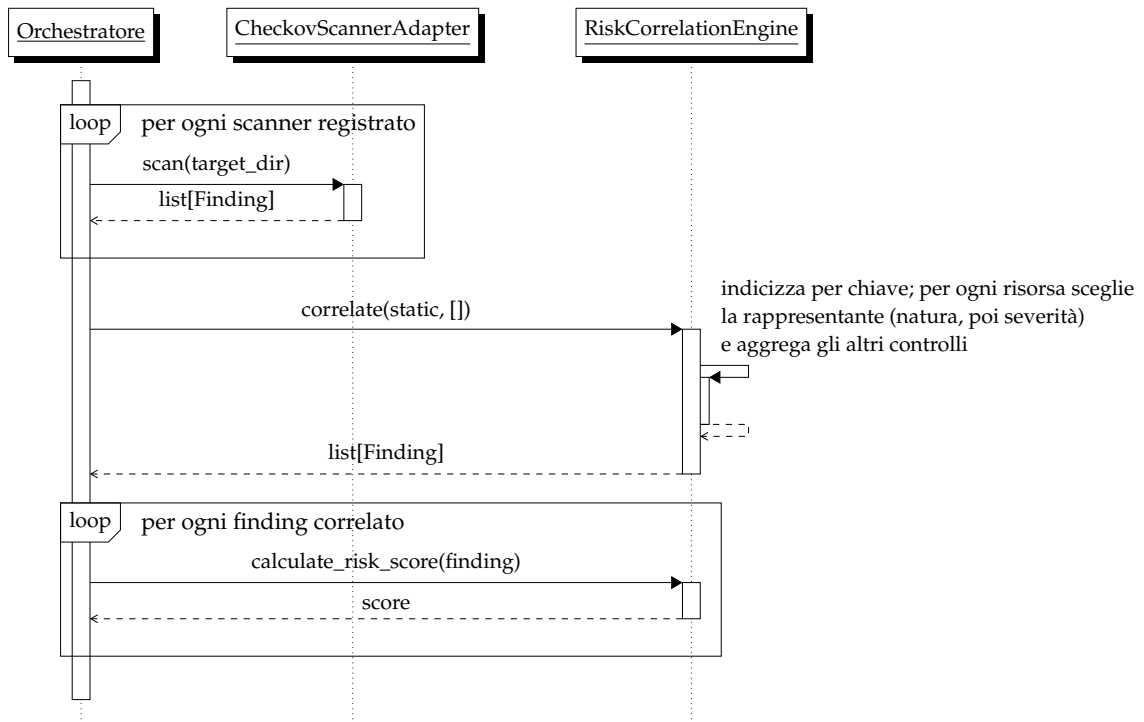


Figura 3.4: Diagramma UML di sequenza della pipeline statica dell’orchestratore. Il secondo argomento di `correlate`, riservato alle segnalazioni a runtime di altri moduli, è vuoto.

Tre termini concorrono al risultato. S è il valore numerico della severità: 10 per CRITICAL, 7,0 per HIGH, 4,5 per MEDIUM, 2,0 per LOW, 0 per INFO. $C \in [0, 1]$ è la confidenza dichiarata dalla sorgente: per Checkov la precisione della regola del catalogo, per ZAP il livello riportato dall’alert, per Semgrep 0,95 se il presidio di autenticazione è stato trovato e 0,7 se la sua assenza è dedotta; sale a 1 quando un riscontro a runtime conferma la segnalazione. X è il contesto, somma dei contributi dichiarati dalla sorgente: 4 per una risorsa raggiungibile da Internet, 2 se è pubblica, 4 se coinvolge dati sensibili, 3 quando nessun contesto è dichiarato. Per gli irrobustimenti X vale sempre 0: una linea di difesa mancante non è un vettore d’accesso. Con CRITICAL pari a 10 il massimo teorico coincide con il limite superiore, che interviene solo quando concorrono tutti i contributi di contesto.

In pratica, per le segnalazioni infrastrutturali (confidenza 1): un’esposizione CRITICAL su risorsa pubblica vale 9,2, una CRITICAL su dati sensibili 8,8, una HIGH senza contesto 6,8, un irrobustimento LOW 3,2. Pesi, punteggi di severità, contributi di contesto e mappe di confidenza risiedono in un file YAML di configurazione; i

valori riportati sono i predefiniti nel codice.

Il cruscotto ordina le voci per natura e poi per punteggio decrescente, mostrato accanto alla severità, e sotto ciascuna elenca i controlli aggregati. Ogni voce e ogni controllo ricevono un'indicazione correttiva: da una base di conoscenza locale indicizzata per controllo oppure, in sua assenza, da un consiglio generato dall'intelligenza artificiale con un modello linguistico eseguito localmente tramite Ollama. Il motore di remediation dipende soltanto dall'interfaccia di dominio `ILlmProvider`; l'adapter di Ollama la realizza e viene iniettato dal punto di composizione, sicché il livello applicativo non conosce l'infrastruttura, come per gli scanner.

3.2.4 Validazione e limiti

Secondo il disegno della Sottosezione 3.1.5, il bersaglio infrastrutturale è costituito da due manifest Terraform: cinque bucket S3 a protezione crescente, dall'ACL pubblica al bucket cifrato con blocco dell'accesso attivo, un sesto bucket con policy aperta, un ruolo IAM con `*` su `*`, una Lambda con chiave cablata e un API Gateway senza autorizzazione. Il confronto previsto dal disegno è quello fra i primi tre bucket, esposti, e gli ultimi due, protetti.

Checkov riporta 86 controlli falliti su 24 risorse Terraform (i bucket sono infatti scomposti nelle rispettive sotto-risorse: blocco dell'accesso pubblico, policy, controlli di proprietà). Tutti i controlli sono risolti dal catalogo per mappatura esplicita, con confidenza 1: 33 sono esposizioni e 53 irrobustimenti. L'ordinamento dei punteggi di rischio (scala 0–10, Sottosezione 3.2.3) rispetta quello atteso, come riportato nella Tabella 3.3.

L'unico scostamento è silenzioso: il controllo `CKV_AWS_70` non valuta la policy aperta del secondo bucket, perché l'attributo `Resource` è una lista di interpolazioni. Il controllo, di conseguenza, non compare né fra i superati né fra i falliti.

Su crAPI [43], revisione 1.1.5, il repository non contiene codice Terraform ma include Dockerfile e manifest Kubernetes. Le 240 segnalazioni prodotte (17 esposizioni e 223 irrobustimenti) si riducono a 46 voci. In testa, con punteggio 7,0, si trovano i token di sessione presenti nella collezione Postman e le credenziali di una dipendenza inclusa nel repository; queste ultime costituiscono un falso positivo, poiché

Risorsa	Punteggio	Natura
ACL pubblica e policy aperta	9,2	esposizione
Chiave cablata (Lambda)	8,8	esposizione
Policy IAM * su *	8,6	esposizione
Metodi senza autorizzazione	7,4	esposizione
Blocchi di accesso disattivati	6,8	esposizione
Bucket sicuri (due)	3,2	solo irrobustimenti

Tabella 3.3: Punteggi di rischio delle voci del bersaglio Terraform, in ordine decrescente.

non appartengono a crAPI. Segue, con 6,8, la specifica OpenAPI priva di requisiti di sicurezza, mentre le voci relative a Kubernetes e Docker chiudono l’inventario con punteggi compresi fra 4,7 e 3,2.

La prova ha inoltre fatto emergere un difetto: l’adapter assegnava la stessa severità a ogni segnalazione, perché le regole operavano sull’identificativo opaco del controllo anziché sul suo nome ufficiale. Il catalogo delle policy descritto nella Sottosezione 3.2.2 nasce proprio da questa correzione.

Restano quattro limiti, nessuno dei quali tocca il criterio di classificazione. Il primo si è già visto: un controllo che non riesce a valutare un’espressione tace anziché fallire, e il silenzio di `CKV_AWS_70` sarebbe passato inosservato senza un riferimento di verità costruito a mano. Il secondo è di manutenzione: il catalogo delle policy va aggiornato a ogni nuova versione di Checkov, altrimenti i controlli aggiunti ricadono sulle regole generiche o sul ripiego predefinito. Il terzo è proprio dell’analisi statica: il manifest mostra ciò che dichiara, non ciò che è effettivamente attivo, e una risorsa modificata a mano dalla console del provider resta invisibile. L’ultimo è il più sottile: il flag di esposizione dice che una risorsa è raggiungibile da chiunque, non se debba esserlo; un bucket che serve i contenuti statici di un sito pubblico viene segnalato esattamente come uno che espone dati riservati.

3.3 Rilevamento della Broken Authentication

La *Broken Authentication* [6] rientra fra le categorie ibride della Sottosezione 3.1.3. Dal codice si capisce che un meccanismo di autenticazione esiste e come è costruito: quali librerie lo implementano, dove si trovano gli endpoint di accesso, quali decoratori e middleware proteggono le rotte e quali dati contiene il token. A volte affiora anche un difetto evidente, come un payload usato senza verificarne la firma. Quello che il codice non dice è come quel meccanismo reagisce: se rifiuta un token falso o scaduto, se chiude la sessione dopo il logout, se frena i tentativi di accesso ripetuti. Sono comportamenti che si osservano solo interrogando il sistema in esecuzione con una richiesta mirata. Per questo la debolezza ha una parte *dinamica* nel senso di RQ_{1.2} e una parte che resta leggibile staticamente.

Il modulo esercita quindi due delle capacità richieste nella Sottosezione 3.1.4 nella stessa pipeline: l'analisi su più livelli, per ricostruire il modello dell'autenticazione a partire da codice, specifica dell'interfaccia ed eventuale traffico osservato, e la verifica dinamica, per accertare la debolezza sul bersaglio attivo. L'esposizione segue questa scansione: prima la costruzione del modello, poi la batteria di sollecitazioni che lo mette alla prova, poi la regola statica che copre il caso in cui il sistema non è interrogabile, e infine la validazione condotta secondo il disegno della Sottosezione 3.1.5.

3.3.1 Ricostruzione del modello di autenticazione

Il modulo non presuppone di conoscere il bersaglio: dato un repository qualsiasi, individua da sé lo stack tecnologico e la gestione dell'identità. Tre fonti statiche indipendenti concorrono a formare un unico modello strutturato, il *grafo di conoscenza dell'autenticazione*, che le fasi successive interrogano.

La prima fonte è una ricognizione dei file dichiarativi del progetto: manifest delle dipendenze (da `package.json` a `requirements.txt` a `pom.xml`), file di configurazione e file di infrastruttura. Su questi un insieme di euristiche deterministiche riconosce linguaggio, framework, librerie di autenticazione, provider d'identità esterno (Keycloak, Auth0, Cognito, Okta) e meccanismi non basati su token come *Basic Auth*, chiavi API o mutua autenticazione TLS. Le stesse informazioni vengono chieste in parallelo a un modello linguistico eseguito localmente tramite Ollama,

che restituisce un oggetto strutturato; quando il modello è irraggiungibile l'analisi ripiega interamente sull'esito delle euristiche, senza interrompersi. Le euristiche hanno comunque la precedenza sui valori dedotti dal modello, perché il risultato resti riproducibile a parità di input. L'intera fase resta peraltro statica e offline: non interroga il bersaglio in esecuzione, ma lavora solo su ciò che il repository dichiara ed espone.

La seconda fonte è l'analisi del codice sorgente. Un parser costruito su *tree-sitter* [46], che copre in modo uniforme otto linguaggi, attraversa ogni file e ne estrae i segnali pertinenti all'autenticazione: import delle librerie individuate, rotte i cui nomi richiamano l'accesso, lettura di variabili d'ambiente sensibili, chiamate alle funzioni delle librerie, funzioni di login e di verifica, claim citati nel codice, decoratori e annotazioni che impongono un ruolo, middleware di protezione. Ogni file riceve un punteggio in base a quanti di questi segnali contiene e viene trattenuto solo oltre una soglia, così da concentrare le fasi seguenti sui pochi file che governano davvero l'identità. Da questa analisi il parser recupera anche, quando è cablato nel codice, il segreto usato per firmare i token: un dettaglio che diventa decisivo nella fase dinamica.

La terza fonte entra in gioco nel motore di correlazione, che fonde i segnali statici con la specifica OpenAPI e con l'eventuale traffico catturato a runtime. Dalla specifica ricava gli schemi di sicurezza dichiarati e le rotte protette; dal traffico decodifica i token realmente osservati e ne estrae claim, ruoli e permessi. Il risultato è il grafo di conoscenza: tipo di autenticazione, provider, endpoint di login, refresh e logout, claim e ruoli, middleware, endpoint protetti, presenza di validazione JWKS, rotazione dei refresh token, indizi di MFA e di limitazione del traffico. A ogni grafo è associato un grado di confidenza compreso fra 0 e 1 che cresce con il numero di fonti concordi: parte da una base minima e aumenta a mano a mano che si aggiungono l'analisi del codice, la specifica dell'interfaccia, il traffico runtime e il riconoscimento di un provider noto. Questo grado è la traduzione operativa della correlazione fra livelli richiesta da $RQ_{1.1}$: nessuna fonte da sola basta a fissare il modello, ma la loro convergenza lo rende via via più completo e affidabile, e più avanti decide quali sollecitazioni abbiano fondamento sufficiente per essere tentate.

3.3.2 La verifica dinamica

Ricostruito il modello, il modulo passa dal potenziale all'accertato sollecitando l'applicazione con richieste reali. Un controllo preliminare di raggiungibilità precede ogni cosa; segue la localizzazione degli endpoint di autenticazione, dopo la quale viene eseguita una batteria di prove asincrone. Ciascuna prova isola una debolezza distinta e restituisce un esito `PASS`, `FAIL` o `INCONCLUSIVE` con severità e categoria interna. La Tabella 3.4 raggruppa le prove per l'aspetto dell'autenticazione che mettono alla prova.

Aspetto verificato	Debolezza sollecitata	Prove
Firma e validazione dei token	manipolazione del token, accettazione di un token scaduto, confusione di algoritmo (RS256→HS256), iniezione di header JWKS	T01, T02, T07, T17
Gestione di sessione e token	riuso dopo il logout, session fixation, riuso e mancata rotazione del refresh token, durata illimitata, abuso in parallelo	T05, T06, T12–T15
Flussi e credenziali	replay del token su un'altra audience, escalation di privilegio dai claim, protezioni dei flussi OAuth2/OIDC (state, PKCE), credenziali non basate su token	T04, T08, T18, T19
Forza bruta e limitazione	assenza di freni sul login, aggiramento del limite tramite spoofing dell'header IP	T03, T20
Autenticazione a più fattori	aggiramento dell'MFA e debolezza dell'OTP	T21
Flussi SAML/SSO	manomissione della firma delle asserzioni	T22
Comportamento e riservatezza	flag di sicurezza dei cookie, informazioni sensibili negli errori, enumerazione degli utenti, reset password deboli	T09, T10, T11, T16

Tabella 3.4: Prove dinamiche del modulo, raggruppate per aspetto dell'autenticazione. La regola statica S01 (Sottosezione 3.3.3) affianca la prima riga per il caso in cui il bersaglio non è interrogabile.

Il legame più stretto fra analisi statica e prova dinamica passa proprio per il segreto di firma recuperato dal codice. Quando è disponibile, la prova sul token scaduto non si limita a inviare un token qualsiasi: ne costruisce uno crittograficamente valido

ma con scadenza nel passato e osserva se il backend lo accetta, distinguendo così un difetto reale di validazione da un semplice rifiuto per firma non riconosciuta. Le prove che presuppongono una sessione autenticata vengono tentate solo se il grado di confidenza del modello supera una soglia predefinita, per non produrre esiti privi di fondamento quando la ricostruzione è incerta. L'esecuzione resta entro guardrail non distruttivi: la prova di forza bruta, per esempio, viene omessa se il bersaglio è dichiarato di produzione. Se poi il target non espone alcun endpoint di login, perché delega il riconoscimento a un provider esterno, le prove che richiederebbero un token non falliscono ma si dichiarano non concludenti: una distinzione che, coerentemente con il disegno della valutazione, tiene separata l'assenza di una debolezza dall'impossibilità di verificarla. A chiusura, gli esiti confluiscono in un punteggio di robustezza sintetico, che parte da cento e viene decurtato per ciascuna prova fallita in proporzione al suo peso. Come le segnalazioni degli altri moduli, anche ogni prova fallita alimenta lo stesso motore di remediation descritto per la parte statica (Sottosezione 3.2.3): a ciascuna debolezza si affianca un'indicazione correttiva, tratta da una base di conoscenza locale o, in sua assenza, generata dall'intelligenza artificiale con un modello linguistico eseguito localmente.

3.3.3 Il caso senza sessione: la regola statica S01

La componente che resta riconoscibile sugli artefatti, indipendentemente dalla possibilità di sollecitare il sistema, è isolata da una regola puramente statica. La regola S01 analizza il codice Python alla ricerca di un pattern ricorrente e insidioso: una funzione che scompone il token sui punti, ne decodifica il payload in Base64 e lo interpreta come JSON, senza mai passare da una libreria di verifica della firma. È il tipico ripiego insicuro adottato quando la validazione crittografica fallisce o il provider è irraggiungibile, e apre la strada a un token forgiato con qualunque identità. Riconoscere i tre elementi entro la stessa funzione basta a segnalare il difetto, ricondotto alla debolezza CWE-347.

Il valore di questa regola sta nel coprire proprio il caso che sfugge alla verifica dinamica: quando l'autenticazione è delegata a un provider esterno e il bersaglio non offre un endpoint di login proprio, le prove sui token non sono eseguibili, ma la

lettura non sicura del payload rimane visibile nel sorgente. È l'esempio concreto del residuo statico che una categoria ibrida conserva anche là dove la sua parte dinamica non è accessibile.

3.3.4 Validazione e limiti

Secondo il disegno della Sottosezione 3.1.5, il primo banco di prova è una coppia di applicazioni gemelle: una versione vulnerabile e una irrobustita che condividono la stessa struttura. La versione vulnerabile concentra cinque debolezze verificate come sfruttabili a mano: un token privo di controllo sull'*audience*, un token scaduto comunque accettato, l'assenza di qualsiasi freno sul login, una session fixation e l'enumerazione degli utenti attraverso messaggi d'errore diversi per utenza esistente e inesistente. Il modulo deve segnalare ciascuna sulla versione vulnerabile e tacere su quella irrobustita, secondo il riferimento della Tabella 3.5. La costruzione del riferimento ha imposto una cautela metodologica: una sesta debolezza inizialmente prevista, l'accettazione dell'algoritmo *none*, è stata esclusa dopo aver constatato che le versioni recenti della libreria di firma la rifiutano a monte, rendendola non sfruttabile; conteggiarla avrebbe gonfiato artificialmente il bersaglio con un difetto inerte.

Debolezza	Endpoint	Vulnerabile	Irrobustita
Nessun controllo dell' <i>audience</i>	/api/profile	atteso	assente
Token scaduto accettato	/api/profile	atteso	assente
Nessuna limitazione sul login	/api/login	atteso	assente
Session fixation	/api/login	atteso	assente
Enumerazione degli utenti	/api/login	atteso	assente

Tabella 3.5: Debolezze attese sulla coppia di applicazioni di prova. Il modulo deve segnalarle sulla versione vulnerabile e non su quella irrobustita.

Il secondo banco di prova è la medesima applicazione realistica usata altrove nel capitolo, OWASP crAPI [43]. Qui la verifica dinamica ha mostrato con chiarezza sia la propria portata sia i propri limiti. Nel primo run il modulo riconosceva una sola

delle debolezze di autenticazione note, l'assenza di limitazione sul login, mentre le prove sulla firma dei token restavano non concludenti. La causa non era nella logica di rilevazione ma nell'accesso al bersaglio: l'helper di login inviava credenziali con chiavi fisse, mentre crAPI richiede il campo *email*, e senza un token di sessione le prove più profonde non potevano partire. Risolta l'acquisizione delle credenziali e imposto il parsing della specifica OpenAPI anche quando il login è già noto, un secondo run ha aggiunto il riconoscimento dell'aggiramento dell'MFA, mentre la famiglia di prove sulla firma dei token è rimasta non concludente per una ragione d'ambiente, ossia l'assenza di utenze di prova registrate in quell'istanza. Quella stessa famiglia, non verificabile dinamicamente, resta coperta staticamente dalla regola S01, che non richiede alcuna sessione.

Questa esperienza mette a fuoco i limiti del modulo, nessuno dei quali intacca la classificazione della categoria. Il più stringente è che la profondità della verifica dipende dalla possibilità di ottenere una sessione autenticata: dove le credenziali non sono disponibili o l'accesso è delegato a terzi, la parte dinamica si ritrae e resta il solo residuo statico. Il riconoscimento dell'MFA e della limitazione del traffico si fonda su parole chiave e librerie note, ed è quindi un'euristica dichiarata, esposta a mancati riconoscimenti su implementazioni fuori dai pattern attesi. L'uso di un modello linguistico nella prima fase introduce una fonte di non determinismo, contenuta dalla precedenza data alle euristiche e dalla degradazione controllata quando il modello è assente. Infine il modulo opera come pipeline autonoma, che produce un proprio rapporto strutturato con il grafo di conoscenza e gli esiti delle prove: una scelta che ne isola la logica ma che tiene le sue segnalazioni distinte dal motore di correlazione descritto per la parte statica.

3.4 Rilevamento della Broken Object Property Level Authorization

La *Broken Object Property Level Authorization* [47], abbreviata in BOPLA, sposta il problema dell'autorizzazione dall'oggetto alla singola proprietà: un utente può avere legittimo accesso a una risorsa e tuttavia leggere o modificare attributi che

non gli competono. Come le altre categorie di autorizzazione, si colloca fra le ibride della Sottosezione 3.1.3, ma con una particolarità che ne segna l’implementazione. Il criterio della Sottosezione 3.1.3 vieta di fornire all’analisi regole puntuali sui permessi delle singole proprietà, perché sarebbe come consegnarle già scritta la risposta che dovrebbe ricavare. Il modulo deve quindi *dedurre* da sé quali attributi appaiano protetti, e solo dopo verificare sul sistema se quella protezione regga davvero. Gli artefatti mostrano l’esistenza e la forma degli oggetti, e talvolta indizi di un controllo su un attributo; se poi l’API applichi quel controllo si scopre solo inviando la stessa richiesta con identità diverse e confrontando le risposte. Il residuo è perciò di nuovo di natura *dinamica* nel senso di RQ_{1,2}.

Il difetto si manifesta in due direzioni indipendenti: una proprietà riservata restituita in lettura a chi non dovrebbe vederla, e una proprietà interna accettata in scrittura da chi non dovrebbe modificarla, il caso noto come *mass assignment*¹. Il modulo le affronta entrambe con una pipeline in quattro fasi coordinata da un orchestratore resiliente agli input parziali: costruisce l’inventario delle proprietà, ne inferisce l’autorizzazione attesa, sollecita il sistema con una batteria di prove mirate e infine aggrega il rischio. L’esposizione segue questa scansione.

3.4.1 Inventario delle proprietà

La prima fase risponde a una domanda preliminare: quali oggetti espone l’applicazione, e con quali attributi. Tre fonti indipendenti concorrono a formare un *inventario delle proprietà*, e per ciascun attributo si registra in quali di esse è comparso. Dalla specifica OpenAPI il modulo legge gli schemi degli oggetti; dal codice sorgente estrae le classi, le interfacce e le strutture che modellano i dati, riconoscendo i tipici oggetti di trasporto² e le entità di persistenza; dal traffico realmente osservato ricava le chiavi presenti nei corpi JSON di richieste e risposte, ricondotte all’oggetto tra-

¹*Mass assignment*: l’assegnazione automatica dei campi ricevuti dal client agli attributi di un oggetto applicativo; se il server non filtra i campi ammessi, il client può valorizzare attributi interni (per esempio `role` o `isAdmin`) non previsti dal flusso normale.

²*DTO (Data Transfer Object)*: oggetto usato per trasferire dati fra il client e il server, spesso una proiezione parziale dell’entità memorizzata nel database.

mite il percorso normalizzato dell'endpoint. L'analisi del codice riusa il parser del modulo precedente (Sottosezione 3.3.1): lo stesso attraversamento sintattico basato su tree-sitter [46], con un ripiego a espressioni regolari quando la grammatica non è disponibile. La normalizzazione dei percorsi degli endpoint, che permette di correlare le tre fonti, è invece affidata a un componente comune a tutta la piattaforma, condiviso con il motore di correlazione e con gli altri rilevatori.

Perché le tre fonti si possano correlare, i nomi degli oggetti vengono ricondotti a una forma comune, eliminando i suffissi convenzionali (`UserDTO` e `UserEntity` diventano entrambi `User`). Così una stessa proprietà osservata nello schema, nel codice e nel traffico viene riconosciuta come unica, e l'inventario tiene traccia della sua provenienza multipla: un dettaglio che la fase successiva usa per pesare quanto quell'attributo sia significativo.

3.4.2 Inferenza dell'autorizzazione per proprietà

Costruito l'inventario, il modulo cerca di capire quali proprietà siano soggette a un controllo di autorizzazione. Non disponendo di una regola esterna, lo deduce da un insieme di indizi che raccoglie per ogni attributo in una scheda di evidenza. Il segnale più diretto è la presenza della proprietà entro un contesto autorizzativo nel codice: un accesso all'attributo che ricade sotto un decoratore di ruolo o dentro una condizione che nomina l'identità del chiamante, il suo ruolo o i suoi permessi. Ad esso si affiancano gli endpoint di lettura e di scrittura che toccano la proprietà, la sua comparsa nello stesso attributo di più modelli distinti, e un segnale di documentazione: un attributo visto nel traffico ma assente dalla specifica è un candidato a proprietà nascosta, il terreno tipico del mass assignment.

Da questi indizi il modulo calcola per ogni proprietà un grado di confidenza compreso fra 0 e 1, sommando contributi fissi per ciascun segnale presente: la comparsa in ognuna delle tre fonti, l'esistenza di endpoint di lettura o di scrittura, la presenza di un contesto autorizzativo, la ricorrenza in più modelli. Il valore così ottenuto è la traduzione operativa della correlazione fra livelli richiesta da RQ_{1.1}: nessun indizio da solo qualifica una proprietà come protetta, ma la loro convergenza la fa emergere e decide quali attributi meritino di essere messi alla prova. Le evidenze confluiscono

in un grafo che associa a ciascun oggetto le sue proprietà con le operazioni di lettura e scrittura e i contesti autorizzativi rilevati.

3.4.3 La verifica dinamica per proprietà

La conferma arriva solo sollecitando il sistema, e per farlo il modulo ha bisogno di due cose: le evidenze della fase precedente e una matrice di identità, ossia le credenziali di un utente ordinario e di un amministratore con cui inviare la stessa richiesta a nome di soggetti diversi. Se l'una o l'altra manca, la fase dinamica non viene eseguita e la pipeline lo dichiara invece di interrompersi. Quando entrambe sono disponibili, una batteria di sette prove mette alla prova le proprietà inferite come protette, cioè quelle con confidenza sopra una soglia o con un contesto autorizzativo noto. La Tabella 3.6 le raggruppa per le due direzioni del difetto.

Direzione	Debolezza sollecitata	Prove
Lettura	un utente ordinario riceve nella risposta una proprietà riservata; confronto differenziale fra ciò che vedono utente e amministratore	T01, T06
Scrittura	un utente ordinario modifica una proprietà protetta, inietta campi non previsti (mass assignment) o attributi non documentati, agisce su proprietà scrivibili ma nascoste dalla lettura; confronto differenziale in scrittura fra utente e amministratore	T02, T03, T04, T05, T07

Tabella 3.6: Le sette prove dinamiche del modulo, raggruppate per la direzione in cui la proprietà attraversa il confine fra client e server. Le prove di scrittura confermano il difetto verificando che il valore alterato sia stato effettivamente persistito con una successiva lettura.

Le prove di lettura ispezionano la risposta: se un attributo giudicato protetto compare fra i dati restituiti a un utente ordinario, oppure se l'amministratore riceve proprietà che all'utente restano invisibili, la discrepanza è registrata. Le prove di scrittura sono più stringenti, perché non si accontentano di un codice di successo: dopo aver tentato di alterare o iniettare una proprietà, rileggono la risorsa e confermano il difetto solo se il valore risulta davvero persistito. Il confronto differenziale fra utente e amministratore consente anche l'esito opposto: quando la modifica dell'utente ordinario viene respinta e quella dell'amministratore va a buon fine, la proprietà è registrata come correttamente protetta. A chiusura, gli esiti si condensano in un

punteggio di robustezza che parte da cento e viene decurtato per ogni categoria di prova confermata, con la relativa fascia di rischio.

3.4.4 Validazione e limiti

La validazione separa ciò che è stato verificato dalle due parti del modulo. Le fasi statiche, inventario e inferenza, sono state esercitate sul repository del progetto e su un target di codice reale, dove hanno censito gli oggetti esposti e le loro proprietà e calcolato per ciascuna il grado di confidenza; la loro correttezza è fissata da una batteria di prove d'unità che verifica il riconoscimento delle classi nei diversi linguaggi, la pulizia dei nomi, la correlazione fra modelli e il calcolo della confidenza. La verifica dinamica, dalla prima all'ultima delle sette prove, è stata validata contro un endpoint vulnerabile controllato che riproduce sia l'esposizione di un attributo riservato sia il mass assignment: su di esso ciascuna prova segnala la propria condizione, mentre la variante irrobustita, che respinge l'utente ordinario e accetta solo l'amministratore, non produce falsi allarmi.

Resta un limite che l'esperienza sul campo ha reso evidente e che conviene dichiarare. Nelle esecuzioni condotte contro applicazioni realmente in funzione, la fase dinamica non è partita per mancanza della matrice di identità, che dev'essere predisposta a parte fornendo le credenziali dei diversi ruoli; in quei casi la pipeline è degradata in modo controllato, riportando l'assenza di rischio anziché interrompersi. La degradazione è una virtù di robustezza, ma nasconde un'ambiguità: un punteggio di robustezza pieno può significare tanto che nessuna debolezza esiste quanto che nessuna è stata verificabile, e i due casi non vanno confusi. La distinzione fra assenza di vulnerabilità e assenza di prova, centrale nel disegno della Sottosezione 3.1.5, richiede quindi di leggere il punteggio insieme al numero di prove effettivamente eseguite.

Gli altri limiti sono più circoscritti. L'inferenza delle proprietà protette si fonda su parole chiave e su contesti autorizzativi riconosciuti nel codice, ed è perciò un'euristica dichiarata, che può non cogliere un controllo espresso in forma inconsueta. Il punteggio di robustezza aggrega per categoria di prova e non per numero di proprietà colpite, e resta quindi una misura grossolana, utile al confronto più che alla

quantificazione. Il modulo è esposto attraverso l'interfaccia di dominio dei rilevatori, coerentemente con l'architettura del sistema, ma le sue segnalazioni conservano la forma orientata alla proprietà con cui vengono prodotte.

3.5 Rilevamento dell'Unrestricted Resource Consumption

L'*Unrestricted Resource Consumption* [13] (Sottosezione 2.2.4) raccoglie i casi in cui manca un limite a governare quanto una richiesta possa consumare: il numero di elementi restituiti, la dimensione di un caricamento, la durata di una chiamata, la frequenza con cui un endpoint costoso può essere invocato. A differenza delle categorie di autorizzazione trattate finora, questa non lascia un residuo che si scopre solo interrogando il sistema. L'assenza di un limite è una proprietà strutturale, leggibile direttamente sugli artefatti di progetto una volta fissate le soglie attese: rientra perciò fra le categorie *tecniche* della Sottosezione 3.1.3, e il modulo che la copre è interamente statico, senza alcuna fase dinamica.

Proprio per questo il modulo è l'esempio più nitido della necessità di analisi su più livelli e di correlazione fra di essi, il nucleo di RQ_{1.1}. Uno stesso difetto, il limite mancante, può risiedere in tre luoghi distinti: nel codice applicativo, nelle definizioni infrastrutturali e nel contratto dell'interfaccia. Nessuno dei tre da solo è sufficiente, perché l'assenza di un limite in un punto cambia significato a seconda di ciò che gli altri prevedono: un caricamento senza controllo di dimensione nel codice non è un difetto se il proxy davanti all'applicazione impone comunque un tetto. Il modulo affronta il problema con tre livelli di analisi che convergono in un unico rapporto; l'esposizione segue questa articolazione.

3.5.1 I tre livelli di analisi

Il primo livello ispeziona il codice sorgente. Attraverso lo stesso attraversamento sintattico basato su tree-sitter [46] già visto per i moduli precedenti, un insieme di sei regole cerca in Python e JavaScript i punti in cui una difesa dovrebbe esistere

e non c'è: una paginazione³ senza tetto sul numero di elementi, un caricamento senza controllo di dimensione, una chiamata di rete priva di timeout⁴, uno schema GraphQL che consente il raggruppamento illimitato di richieste in una sola, un ciclo che itera su un input controllato dall'utente, la chiamata a un servizio esterno a pagamento senza alcun freno di frequenza.

Il secondo livello guarda ai file di configurazione, quelli che governano l'infrastruttura e non la logica applicativa, e non usa il parser sintattico ma letture mirate per formato. Verifica se `docker-compose.yml` imponga un tetto di memoria ai servizi, se la configurazione del proxy o le impostazioni dell'applicazione fissino una dimensione massima del corpo delle richieste, se il server web e quello applicativo definiscano un timeout: un valore di timeout azzerato, che disabilita il controllo, viene trattato come il difetto più grave, mentre un timeout semplicemente non dichiarato, per cui vale un valore predefinito ragionevole, riceve una severità minore.

Il terzo livello, eseguito solo quando è disponibile, legge il contratto OpenAPI dell'interfaccia. Vi cerca le garanzie che dovrebbero essere dichiarate prima ancora che l'API entri in funzione: un parametro di paginazione privo di valore massimo, un endpoint di caricamento il cui schema non fissa una lunghezza o un numero massimo di elementi⁵, un endpoint riconosciuto come costoso, per esempio un invio di SMS o una generazione di report, che non documenta alcuna protezione di frequenza né richiede autenticazione. Facoltativamente, questo livello può annotare il contratto stesso, inserendo accanto a ogni operazione interessata un campo di estensione che riporta l'esito dell'analisi, senza alterare i campi esistenti.

Ogni segnalazione, da qualunque livello provenga, ha la stessa forma: la regola che l'ha prodotta, la debolezza CWE associata (prevalentemente CWE-400 [48] e CWE-770 [17], con la frequenza di interazione di CWE-799 [49]), la severità, un grado di confidenza, il livello di origine e soprattutto la descrizione della *difesa mancante*,

³*Paginazione*: la suddivisione di una risposta voluminosa in pagine di dimensione limitata, così che una singola richiesta non restituisca un'intera collezione.

⁴*Timeout*: il tempo massimo concesso a un'operazione prima di interromperla; in sua assenza una chiamata lenta può restare appesa a tempo indefinito, trattenendo risorse.

⁵Nel contratto OpenAPI i vincoli `maximum`, `maxLength` e `maxItems` dichiarano rispettivamente il valore, la lunghezza e il numero di elementi massimi ammessi per un campo.

ossia del controllo che ci si aspetterebbe di trovare. La Tabella 3.7 riassume le dodici regole distribuite sui tre livelli.

3.5.2 Aggregazione e correlazione fra livelli

I tre livelli vengono eseguiti in sequenza e i loro esiti confluiscono in un unico rapporto. Il codice e la configurazione sono sempre analizzati; il contratto solo se una specifica viene fornita, altrimenti quel livello è semplicemente omesso. Il rapporto non è un semplice elenco: accanto alle segnalazioni raccoglie i *segnali di copertura*, che per ogni categoria di difetto indicano quali regole si sono attivate, e un riepilogo che conta le segnalazioni per severità, per categoria e per livello. È questa vista unificata a rispondere alla correlazione richiesta da RQ_{1.1}: una stessa categoria, come l'assenza di un tetto alla dimensione dei dati, si presenta al lettore attraverso i livelli in cui quel tetto avrebbe dovuto essere imposto, dal codice al contratto.

Le soglie oltre le quali un limite dichiarato è comunque giudicato eccessivo, o un valore di timeout è considerato pericoloso, sono parametri espliciti dell'analisi e non costanti nascoste nelle regole, in accordo con la configurabilità richiesta dalla Sottosezione 3.1.4. Prima dell'aggregazione le segnalazioni duplicate, riferite alla stessa regola sullo stesso punto, vengono ridotte a una sola, mantenendo quella con confidenza maggiore, e l'insieme è ordinato per severità decrescente.

3.5.3 Validazione e limiti

La validazione segue il disegno della Sottosezione 3.1.5 con una coppia di applicazioni gemelle: una versione vulnerabile che riunisce i difetti attesi ai livelli del codice e della configurazione, e una versione irrobustita che li corregge tutti mantenendo la stessa struttura. Il riferimento di verità prevede che la prima faccia scattare ogni regola applicabile e che la seconda non ne faccia scattare nessuna. Nel ciclo di validazione registrato il modulo ha riconosciuto tutte le debolezze presenti nella versione vulnerabile e non ha prodotto alcuna segnalazione su quella irrobustita, con un tasso di veri positivi del cento per cento e nessun falso positivo. La regola sul raggruppamento GraphQL, non esercitabile su applicazioni che non usano quella tecnologia, è validata separatamente da prove d'unità dedicate; allo stesso modo le

Livello	Regole	Difesa mancante ricercata
Codice (AST)	RC-001-006	tetto di paginazione, dimensione del caricamento, timeout delle chiamate, raggruppamento GraphQL, limiti dei cicli su input, freno sui servizi esterni a pagamento
Configurazione	RC-007-009	tetto di memoria dei container, dimensione massima del corpo, timeout di richiesta del proxy e del server applicativo
Contratto (OpenAPI)	RC-010-012	valore massimo dei parametri di paginazione, limiti di dimensione dei caricamenti, protezione degli endpoint costosi

Tabella 3.7: Le dodici regole del modulo, distribuite sui tre livelli di analisi. Ciascuna descrive una difesa attesa contro il consumo incontrollato e, se assente, produce una segnalazione.

tre regole sul contratto OpenAPI sono verificate con specifiche costruite ad hoc nei test, non attraverso i file della coppia gemella.

Restano alcuni limiti dichiarati. Il riconoscimento sul codice JavaScript è parziale, perché una variazione nella grammatica di tree-sitter lascia scoperti alcuni pattern che sul codice Python vengono invece individuati; la copertura resta quindi più solida per quest'ultimo. Il modulo segnala l'assenza di un limite, ma non conosce la soglia appropriata al singolo contesto applicativo: può dire che un tetto manca o che un valore dichiarato appare fuori scala, non quale sia il valore giusto, che resta un parametro da dichiarare. La correlazione fra livelli, inoltre, è di natura presentativa: il rapporto affianca i segnali dei tre livelli e ne offre una lettura d'insieme, ma non annulla automaticamente una segnalazione del codice quando la difesa corrispondente è garantita altrove, nella configurazione o nel contratto; questo giudizio resta in parte affidato a chi legge. Infine, coerentemente con la natura della categoria, le segnalazioni riguardano difese mancanti, presidi di robustezza la cui assenza non costituisce di per sé un varco immediatamente sfruttabile: la severità misura l'importanza del limite, non la certezza di un attacco riuscito.

3.6 Rilevamento della Broken Function Level Authorization

La *Broken Function Level Authorization* [23] (Sottosezione 2.2.5) riguarda le funzioni che un ruolo non dovrebbe poter invocare: esportare l'elenco degli utenti, cancellare un account altrui, creare un invito con privilegi amministrativi. Si distingue dalla BOLA per il livello a cui il controllo cede: non l'oggetto ma la funzione. Il difetto è insidioso perché il controllo di autorizzazione a livello di funzione non ha un luogo obbligato, e una rotta aggiunta a un controller eredita il routing e l'autenticazione ma non l'autorizzazione, nascendo raggiungibile e restando tale. La sua parte riconoscibile sugli artefatti è l'assenza, su una funzione riservata, del controllo che ne verifichi il ruolo; accertare poi che quell'assenza sia davvero sfruttabile da un'utenza ordinaria è un residuo che richiederebbe una sollecitazione dinamica, non compiuta da questo modulo. Il rilevatore adotta perciò lo stesso impianto statico a tre livelli

del modulo precedente, con un problema in più da governare: uno stesso endpoint può essere segnalato da più regole e da più livelli, e va ricondotto a un'unica voce. L'esposizione tratta prima i tre livelli, poi la deduplicazione che li unifica, infine la validazione.

3.6.1 I tre livelli e la mappatura degli endpoint

Il primo livello analizza il codice sorgente e, a differenza degli altri moduli, non si limita a cercare i difetti: mappa attivamente ogni funzione all'endpoint che espone, ricostruendo il percorso dalle rotte dichiarate nei decorator e dai prefissi dei raggruppamenti di rotte. Su questa mappa sei regole cercano le diverse forme in cui il controllo di funzione può mancare o essere aggirato: un endpoint privilegiato privo di qualsiasi verifica di ruolo, il caso più grave e ricondotto a CWE-285 [24]; un endpoint che autentica il chiamante ma non ne verifica i privilegi, ossia autenticazione senza autorizzazione (CWE-862 [50]); un verbo HTTP⁶ alternativo lasciato aperto su una rotta protetta solo per il verbo consueto (CWE-650 [51]); un percorso amministrativo raggiungibile senza restrizioni (CWE-284 [52]); una funzione amministrativa collocata fuori dai percorsi riservati, e perciò sfuggente a un controllo basato sul solo nome della rotta (CWE-285).

Il secondo livello ispeziona i file di configurazione alla ricerca del principio che il capitolo precedente ha indicato come la difesa centrale di questa categoria: il *diniego per impostazione predefinita*⁷. Una configurazione che conceda accesso per impostazione predefinita, anziché negarlo e richiedere permessi espliciti, viene segnalata come un default permissivo (CWE-276 [53]).

Il terzo livello, presente solo se una specifica OpenAPI è fornita, incrocia i risultati dell'analisi del codice con il contratto: cerca gli endpoint privilegiati che la speci-

⁶Verbo (o metodo) HTTP: l'azione richiesta su una risorsa, per esempio GET per leggere o DELETE per cancellare; lasciarne abilitato uno non presidiato riapre l'operazione da una via secondaria.

⁷*Deny-by-default*: il criterio per cui ogni funzione è negata salvo concessione esplicita al ruolo che ne ha diritto; con il criterio opposto, il permesso predefinito, una dimenticanza non produce alcun segnale d'errore e passa inosservata.

fica dichiara senza uno schema di sicurezza⁸, e produce segnalazioni che portano l'evidenza congiunta dei due livelli. La Tabella 3.8 riepiloga le sei regole.

Regola	Debolezza ricercata	Severità
BF-001	endpoint privilegiato privo di controllo di ruolo	CRITICAL
BF-002	autenticazione presente ma autorizzazione assente	HIGH
BF-003	verbo HTTP alternativo lasciato non presidiato	HIGH
BF-004	percorso amministrativo esposto senza restrizioni	HIGH
BF-005	default permissivo anziché di diniego (configurazione)	MEDIUM
BF-006	funzione amministrativa fuori dai percorsi riservati	HIGH

Tabella 3.8: Le sei regole del modulo. BF-005 opera sui file di configurazione, le altre sul codice e, quando disponibile, sul contratto OpenAPI.

3.6.2 Deduplicazione per priorità di regola

L'analisi a tre livelli comporta un rischio di ridondanza: la mancanza di un controllo su una rotta può essere rilevata insieme dall'esame del codice e da quello del contratto, e più regole possono colpire lo stesso endpoint da angoli diversi. Segnarli tutti separatamente frammenterebbe il difetto e ne gonfierebbe il conteggio. Il modulo riconduce perciò ogni endpoint a un'unica voce rappresentativa, scelta attraverso un ordine di priorità fra le regole. La priorità non coincide con la severità, ma esprime quale caratterizzazione sia più specifica per quell'endpoint: una funzione amministrativa fuori posto o un percorso amministrativo esposto descrivono il difetto in modo più puntuale del generico endpoint privo di controllo, e prevalgono su di esso. A parità di priorità, il modulo preferisce la segnalazione ancorata a una riga di codice concreta, prodotta dall'analisi del sorgente, rispetto a quella che rimanda

⁸*Schema di sicurezza (security scheme)*: la sezione del contratto OpenAPI che dichiara come un endpoint è protetto, per esempio tramite token di bearer o chiave API; la sua assenza su una funzione riservata segnala che il contratto non prevede alcuna protezione.

solo al contratto: la prima indica dove intervenire, riducendo il lavoro di bonifica. Le segnalazioni di configurazione, che non si riferiscono a un singolo endpoint, restano fuori da questa competizione e vengono conservate a parte. L'insieme così ridotto confluisce in un rapporto con i segnali di copertura e il riepilogo per severità, categoria e livello, con la possibilità di annotare il contratto OpenAPI in luogo, come per il modulo precedente.

3.6.3 Validazione e limiti

La validazione riprende il disegno a coppia gemella: un'applicazione vulnerabile che riunisce le sei debolezze e una versione irrobustita che le corregge tutte. Il riferimento di verità richiede che la prima faccia scattare ciascuna delle sei regole e che la seconda non ne faccia scattare nessuna. Nel ciclo registrato il modulo ha riconosciuto tutte e sei le debolezze sulla versione vulnerabile e non ha prodotto alcuna segnalazione su quella irrobustita, con un tasso di veri positivi del cento per cento e nessun falso positivo.

I limiti discendono dalla natura statica dell'analisi e dalle euristiche che la guidano. Il riconoscimento di un endpoint come privilegiato si fonda su parole chiave nei percorsi e nei nomi delle funzioni: una funzione riservata con un nome anodino può sfuggire, e per converso un endpoint dal nome allarmante ma legittimo può essere segnalato a torto. Soprattutto, il modulo verifica la presenza del controllo di funzione, non la sua efficacia: accertare che un controllo dichiarato blocchi davvero un'utenza ordinaria richiederebbe di inviare la richiesta a nome di quell'utenza e osservarne l'esito, la sollecitazione dinamica che qui non viene compiuta e che resta il residuo della categoria. Il riconoscimento sul codice JavaScript è parziale come nel modulo precedente, più solido su Python. Infine la deduplicazione, tenendo un'unica regola per endpoint, subordina le debolezze secondarie che insistono sulla stessa rotta alla voce rappresentativa: una scelta che favorisce la leggibilità e la bonifica, al prezzo di non enumerare separatamente ogni difetto coincidente.

3.7 Rilevamento della Server-Side Request Forgery

3.7.1 Metodo di ricerca e sistema di rilevazione

La metodologia adottata per la rilevazione delle vulnerabilità di tipo Server-Side Request Forgery (SSRF) si basa sulla progettazione e implementazione di un modulo di analisi integrato nella piattaforma di rilevamento dei rischi di sicurezza delle API. L'obiettivo del sistema è individuare condizioni di esposizione riconducibili a OWASP API7:2023 e CWE-918, concentrandosi in particolare sui casi in cui un endpoint utilizza dati controllati dall'utente per costruire richieste verso risorse remote senza applicare controlli sufficienti sulla destinazione. Il metodo combina l'analisi del codice sorgente con l'analisi del contratto dell'API, in modo da osservare il problema da due prospettive complementari: il comportamento effettivamente implementato e i vincoli dichiarati nell'interfaccia. Tale scelta permette di non limitare il rilevamento alla presenza di specifiche chiamate di rete, ma di considerare anche il percorso seguito dai dati prima che raggiungano tali operazioni.

Il sistema è organizzato all'interno del modulo `src/core/api7_ssrf/`, che costituisce il componente responsabile dell'analisi relativa alla SSRF. La struttura è suddivisa in più componenti, ciascuno associato a una fase distinta del processo di rilevamento. Il punto di ingresso è rappresentato da `detector.py`, che coordina l'esecuzione delle diverse analisi e raccoglie i risultati in un report comune. La gestione dei risultati è affidata ai modelli definiti in `models.py`, mentre `semgrep_runner.py` si occupa dell'esecuzione controllata del motore Semgrep. L'elaborazione dei risultati prodotti dall'analisi statica è demandata a `normalizer.py`, che ricostruisce le informazioni principali del flusso rilevato e le rende omogenee rispetto al modello interno. La componente `layer3_openapi.py` completa il processo analizzando le specifiche OpenAPI 3.x. Questa separazione consente di mantenere distinte le responsabilità delle diverse fasi, facilitando sia la gestione dei risultati sia l'estensione del sistema con ulteriori controlli.

Il nucleo dell'approccio è costituito dall'analisi statica mediante Semgrep, configurata attraverso regole specifiche per il rilevamento della SSRF. A differenza di una ricerca basata esclusivamente sulla presenza di determinate funzioni o stringhe

nel codice, il sistema considera la propagazione dei dati attraverso l'applicazione. In particolare, vengono individuate le sorgenti dalle quali possono provenire dati controllati dall'utente, come parametri di query, corpo della richiesta, parametri del percorso e intestazioni HTTP, e viene seguito il loro percorso fino alle operazioni che effettuano richieste di rete. Quando un dato non adeguatamente controllato raggiunge una di queste destinazioni, il flusso viene considerato potenzialmente rischioso. Questo modello permette quindi di mettere in relazione l'origine del dato con il punto in cui esso viene utilizzato, fornendo un'informazione più significativa rispetto alla semplice individuazione di una chiamata HTTP.

Nel processo di analisi vengono riconosciuti anche i controlli che possono interrompere il flusso di contaminazione. Il sistema considera, ad esempio, funzioni utilizzate per verificare la validità di un URL o per controllare che la destinazione appartenga a un insieme di host autorizzati. La presenza di tali controlli viene utilizzata per distinguere un flusso non protetto da uno nel quale sono state introdotte misure di validazione. Questa distinzione è rilevante per limitare le segnalazioni non necessarie e, allo stesso tempo, per fornire una descrizione più precisa della condizione individuata. L'analisi tiene conto anche dei casi in cui il valore controllato dall'utente costituisce soltanto una parte dell'URL finale, ad esempio quando viene inserito all'interno di una stringa predefinita mediante concatenazioni, *f-string* o formattazione di stringhe. In questo modo il rilevatore può considerare anche scenari di *Partial SSRF*, nei quali la destinazione non è interamente determinata dall'input ma quest'ultimo mantiene comunque un'influenza sul percorso della richiesta.

A supporto dell'analisi del flusso sono state definite sei regole di rilevamento, identificate come SS-001–SS-006. Le prime quattro riguardano principalmente il codice applicativo e coprono i client HTTP più comuni nei linguaggi considerati. SS-001 analizza l'utilizzo di `requests`, `httpx` e `aiohttp` in Python, mentre SS-002 estende il controllo alle funzioni di `urllib`. SS-003 riguarda le applicazioni Node.js ed Express e verifica la propagazione degli input provenienti dalla richiesta verso `fetch` e `axios`. SS-004 prende invece in considerazione l'abilitazione dei reindirizzamenti automatici nelle richieste HTTP, poiché una destinazione inizialmente verificata può essere modificata attraverso una catena di `redirect`. La regola SS-005 utilizza un controllo basato su pattern per individuare richieste dirette verso endpoint associati

ai servizi di metadati cloud, mentre SS-006 opera a livello di specifica OpenAPI. L'insieme delle regole consente così di coprire differenti modalità attraverso le quali una stessa classe di vulnerabilità può manifestarsi.

Una seconda componente della metodologia riguarda infatti l'analisi contrattuale dell'API. La specifica OpenAPI viene esaminata per individuare parametri e proprietà del corpo delle richieste che possono rappresentare destinazioni URL, con particolare attenzione a campi quali `webhook`, `callback_url`, `redirect` ed `endpoint`. La verifica considera la presenza di vincoli che descrivano il valore atteso, come `format: uri`, e di eventuali pattern utilizzati per limitare le destinazioni ammesse. L'obiettivo non è stabilire direttamente che l'implementazione sia vulnerabile, ma verificare se il contratto dell'interfaccia fornisca indicazioni sufficienti a restringere l'insieme dei valori accettabili. Questa fase permette quindi di individuare una possibile debolezza già a livello progettuale, prima di considerare il comportamento dell'applicazione durante l'esecuzione.

I risultati provenienti dalle diverse fasi vengono successivamente normalizzati in una rappresentazione comune. Il normalizzatore estrae dal risultato di Semgrep le informazioni relative alla sorgente e alla destinazione del flusso e associa al reperto una valutazione della confidenza. Quest'ultima tiene conto della relazione osservata tra input e operazione di rete e della presenza di eventuali controlli intermedi. Il sistema distingue anche le strategie di validazione riconosciute, in particolare i controlli basati su *allowlist*, quelli basati su *blocklist* e i casi nei quali non viene individuata alcuna protezione. Il risultato finale viene raccolto in una struttura di report che conserva, insieme alla segnalazione, le informazioni necessarie per interpretarne l'origine e il livello di affidabilità.

La validazione del metodo è stata condotta utilizzando casi di test appositamente predisposti, distinguendo applicazioni vulnerabili da applicazioni nelle quali erano presenti meccanismi di protezione. I casi vulnerabili comprendono differenti modalità di utilizzo dei client HTTP, richieste asincrone, utilizzo di `urllib`, `redirect` attivi e accesso ai servizi di metadati cloud. I casi protetti includono invece controlli sugli URL, verifica dello schema `https`, restrizioni mediante *allowlist* degli hostname e disabilitazione dei `redirect`. Il sistema ha inoltre previsto una verifica su un repository reale, nella quale un endpoint di importazione è stato correttamente individuato

come caso positivo mentre una variante protetta dello stesso flusso non ha prodotto una segnalazione. La validazione è quindi stata impostata in modo da considerare sia la capacità del rilevatore di riconoscere i casi vulnerabili sia la capacità di evitare segnalazioni nei percorsi esplicitamente protetti.

Nel complesso, il metodo adottato combina quindi analisi del flusso dei dati, riconoscimento di specifici comportamenti rischiosi e verifica dei vincoli dichiarati a livello di API. La scelta di integrare questi livelli deriva dalla necessità di osservare la SSRF non come una singola istruzione vulnerabile, ma come una condizione che emerge dalla relazione tra input, controlli e destinazione della richiesta. Il sistema mantiene tuttavia i limiti propri di un approccio prevalentemente statico: non può verificare direttamente la raggiungibilità della risorsa a runtime né confermare un'eventuale esfiltrazione fuori banda tipica delle *Blind SSRF*. Restano inoltre fuori dal tracciamento locale i flussi che attraversano componenti distribuiti, come code o broker di messaggi, prima di raggiungere il client HTTP. Questi aspetti rappresentano possibili direzioni di estensione del metodo attraverso tecniche di analisi dinamica e meccanismi di correlazione tra componenti distribuiti.

3.8 Rilevamento della Security Misconfiguration

La *Security Misconfiguration* [25] (Sottosezione 2.2.8) raccoglie errori di configurazione che possono verificarsi a diversi livelli, dalla rete fino all'applicazione. Questa sezione ne affronta la parte visibile nel codice sorgente e nei file di progetto, completando l'analisi infrastrutturale già trattata nella Sezione 3.2. Il modulo adotta un approccio statico: cerca direttamente nel codice le scelte di configurazione insicure, senza richiedere l'avvio o l'esecuzione del software.

3.8.1 Regole locali e regole globali

Lo scanner analizza i file sorgente in Python, JavaScript e TypeScript, escludendo le directory non applicative (come ambienti virtuali o cartelle di build). I controlli sono articolati in verifiche locali su ciascun file e verifiche globali sul progetto (Tabella 3.9).

A livello di singolo file, il modulo intercetta i comportamenti a rischio più comuni: politiche CORS⁹ permissive o aperte assieme alle credenziali utente, modalità di debug e gestori di errore che espongono la traccia dello stack, segreti cablati nel codice individuati tramite entropia di Shannon¹⁰ e route prive di restrizioni sui metodi HTTP o delle necessarie direttive `Cache-Control`.

A livello globale, tre verifiche valutano l'impostazione complessiva del servizio: l'assenza delle intestazioni HTTP difensive raccomandate (come HSTS e `Content-Security-Policy`), la totale mancanza di una politica CORS e la presenza di dipendenze con vulnerabilità note censite in un archivio locale.

Regola	Comportamento ricercato	Ambito	Severità
SC-001	politica CORS con carattere jolly	file	HIGH
SC-002	modalità di debug attiva	file	HIGH
SC-003	gestore d'errore che espone la traccia dello stack	file	HIGH
SC-004	header di sicurezza mancanti	globale	MEDIUM
SC-005	segreto cablati nel codice (entropia di Shannon)	file	CRITICAL
SC-006	assenza totale di politica CORS applicativa	globale	MEDIUM
SC-007	dipendenze applicative con vulnerabilità note	globale	HIGH
SC-008	metodi HTTP non restritti o route mutanti su GET	file	MEDIUM
SC-009	assenza di <code>Cache-Control</code> su route con dati sensibili	file	MEDIUM

Tabella 3.9: Il catalogo delle regole del modulo di rilevamento applicativo per API8:2023.

⁹CORS (*Cross-Origin Resource Sharing*): meccanismo con cui un server dichiara quali siti di origine diversa possono leggere le sue risposte dal browser; una configurazione permissiva consente a domini terzi non autorizzati di accedere a dati riservati.

¹⁰*Entropia di Shannon*: misura del grado di casualità di una stringa; sequenze ad alto disordine indicano chiavi o token reali, distinguendoli dai normali identificatori.

3.8.2 Limiti del modulo

I limiti del modulo dipendono dalla sua natura statica. Non analizzando il traffico reale o il comportamento a runtime, lo strumento non può individuare problemi legati ai server web o ai proxy intermedi, come la desincronizzazione delle richieste HTTP o l'abilitazione di metodi pericolosi a livello server (come il verbo `TRACE`). Inoltre, il controllo delle librerie fa riferimento a un archivio locale offline, lasciando l'aggiornamento continuo a strumenti dedicati.

3.9 Rilevamento dell'Unsafe Consumption of APIs

Le categorie viste finora guardano ai dati che l'API riceve dai suoi utenti. L'*Unsafe Consumption of APIs* [42] (Sottosezione 2.2.10) ribalta il punto di vista: qui il pericolo arriva dai servizi di terze parti da cui l'applicazione stessa dipende, come un gateway di pagamento o un fornitore di dati esterni. Avendoli scelti e configurati, l'applicazione tende a fidarsi delle loro risposte senza troppe domande; ma quelle risposte attraversano pur sempre un confine di fiducia¹¹ che spesso nessuno sorveglia. OWASP elenca cinque forme di questa fiducia mal riposta: canale non cifrato, dati non validati, redirezioni seguite ciecamente, risorse senza limiti e timeout assenti; per almeno tre di esse si è osservato che ricadono già in altre categorie [54].

Il modulo si occupa delle tre forme tipiche del consumo insicuro (canale in chiaro, dati non validati e redirezioni cieche) e lascia le altre due, l'assenza di limiti e di timeout, al modulo dell'Unrestricted Resource Consumption (Sezione 3.5), che ne è la sede naturale; così lo stesso difetto non viene contato due volte, come chiede il criterio della Sottosezione 3.1.3. Le tre forme rimaste si leggono direttamente nel codice: nella parte che lo strumento accerta la categoria si comporta come una *tecnica*, e il rilevatore è statico, lavora a un solo livello e analizza sorgenti Python. Ciò che sfugge all'analisi statica, ossia se la predisposizione al consumo insicuro sfoci in una

¹¹*Confine di fiducia (trust boundary)*: la linea che separa la parte di un sistema di cui ci si fida da quella che va invece verificata; collocarlo nel punto sbagliato significa accettare senza controlli dati che provengono in realtà dall'esterno.

compromissione reale, resta però un residuo di natura *dinamica* nel senso di RQ_{1,2}, ripreso e classificato in sede di validazione (Sottosezione 3.9.2).

3.9.1 Le tre regole

Uno scanner percorre i file Python del bersaglio, li legge con l'analizzatore sintattico della libreria standard e applica tre regole, riassunte nella Tabella 3.10. La prima insegue il dato che arriva da un servizio esterno e viene poi accettato, salvato o passato ad altri componenti senza alcun controllo: è l'iniezione differita, dove un contenuto malevolo entra dal canale, ritenuto fidato, di una terza parte e fa danni più tardi, quando l'applicazione lo rilegge e lo usa (CWE-20 [55]). La seconda cerca le chiamate che raggiungono un servizio esterno su canale in chiaro¹², cioè con `http` invece di `https` (CWE-319 [56]). La terza individua i client che seguono ciecamente le redirezioni¹³ indicate da una terza parte (CWE-601 [57]).

Regola	Comportamento ricercato	Severità
UC-001	dati di terze parti usati senza validazione	HIGH
UC-002	consumo di un servizio esterno su canale in chiaro (<code>http</code>)	HIGH
UC-003	redirezioni di terze parti seguite ciecamente	HIGH

Tabella 3.10: Le tre regole del modulo, corrispondenti alle condizioni di consumo insicuro non già coperte dalla Sezione 3.5.

Come nel modulo precedente, ogni segnalazione porta con sé un grado di confidenza: quelle sotto la soglia predefinita di 0,70 vengono scartate prima di comporre il rapporto, che riunisce i segnali di copertura e il riepilogo per severità e categoria.

¹²In chiaro (*cleartext*): senza cifratura; un URL `http://` anziché `https://` espone i dati in transito a chi intercetta il traffico, l'attacco *man-in-the-middle*.

¹³Redirezione (*redirect*): la risposta con cui un server rimanda il client a un altro indirizzo; seguirla in automatico, con l'opzione `allow_redirects` attiva, consente a un fornitore compromesso di dirottare la richiesta, dati sensibili compresi, verso un server ostile.

3.9.2 Validazione e limiti

Sulle due applicazioni gemelle il risultato è netto: le tre regole scattano tutte sulla versione vulnerabile e nessuna su quella irrobustita, con il cento per cento di veri positivi e zero falsi positivi. Il modulo dimostra quindi di rilevare in modo affidabile i pattern di consumo insicuro per i quali è stato progettato.

Sull'applicazione realistica le regole non producono segnalazioni, un esito che riflette le caratteristiche architetturali del codice esaminato piuttosto che un difetto di copertura. Le regole UC-002 e UC-003 identificano il servizio esterno a partire da un indirizzo statico presente nel sorgente; nell'applicazione realistica, invece, gli endpoint delle terze parti sono definiti solo a runtime, perciò non rientrano nel perimetro dell'analisi puramente statica. La regola UC-001, che non dipende dall'indirizzo ma segue il flusso del dato dalla risposta della terza parte fino al punto di riuso, non genera segnalazioni perché il codice esaminato non presenta punti di iniezione non protetti, come query costruite per concatenazione o oggetti creati da dizionari non filtrati. In questo caso è dunque assente il bersaglio dell'iniezione differita, non la capacità di rilevamento del modulo. Lo stesso codice resta inoltre coperto, da un'angolazione complementare, dal modulo per la Server-Side Request Forgery (Sezione 3.7).

I risultati confermano la collocazione naturale del modulo all'interno di un'architettura di analisi a più livelli. L'analisi statica individua la *predisposizione* al consumo insicuro — la validazione assente, il canale in chiaro, la redirectione seguita ciecamente — mentre la verifica di sfruttabilità effettiva richiede un piano dinamico che metta alla prova il sistema in esecuzione contro un fornitore ostile e controllato: un banco che osservi il traffico in uscita e risponda di proposito con dati malevoli o con una redirectione verso un host pericoloso. API10 presenta dunque una copertura *mista* — statica per la predisposizione, dinamica e integrata con l'SSRF per la verifica — e il residuo non coperto rientra nella tassonomia della RQ_{1.2}.

CAPITOLO 4

Conclusioni

Le conclusioni qui...

Bibliografia

- [1] OWASP Foundation, "A05:2021 – security misconfiguration," https://owasp.org/Top10/2021/A05_2021-Security_Misconfiguration/, 2021, oWASP Top 10:2021. [Online]. Available: https://owasp.org/Top10/2021/A05_2021-Security_Misconfiguration/ (Citato alle pagine 4 e 5)
- [2] SentinelOne, "Cloud misconfigurations: Risks, examples & prevention," <https://www.sentinelone.com/cybersecurity-101/cloud-security/cloud-misconfigurations/>, 2025. [Online]. Available: <https://www.sentinelone.com/cybersecurity-101/cloud-security/cloud-misconfigurations/> (Citato a pagina 5)
- [3] Cloud Computing News, "10 real-life cloud security failures and what we can learn from them," <https://www.cloudcomputing-news.net/news/10-real-life-cloud-security-failures-and-what-we-can-learn-from-them/>, 2025. [Online]. Available: <https://www.cloudcomputing-news.net/news/10-real-life-cloud-security-failures-and-what-we-can-learn-from-them/> (Citato a pagina 5)
- [4] OWASP Foundation, "OWASP API security top 10 – 2023," <https://owasp.org/API-Security/editions/2023/en/0x11-t10/>, 2023. [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0x11-t10/> (Citato alle pagine 7, 17 e 38)

- [5] OWASP API Security Project, “API1:2023 broken object level authorization,” <https://owasp.org/API-Security/editions/2023/en/0xa1-broken-object-level-authorization/>, 2023, oWASP API Security Top 10 – 2023. Consultato il 25 agosto 2026. [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0xa1-broken-object-level-authorization/> (Citato alle pagine 8 e 22)
- [6] —, “API2:2023 Broken Authentication,” <https://owasp.org/API-Security/editions/2023/en/0xa2-broken-authentication/>, 2023, consultato il 25 agosto 2026. [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0xa2-broken-authentication/> (Citato alle pagine 15, 16, 21 e 51)
- [7] OWASP Foundation, “Credential Stuffing,” https://owasp.org/www-community/attacks/Credential_stuffing, oWASP Community Pages. [Online]. Available: https://owasp.org/www-community/attacks/Credential_stuffing (Citato alle pagine 15 e 16)
- [8] Y. Sheffer, D. Hardt, and M. Jones, “JSON Web Token Best Current Practices,” RFC 8725, Internet Engineering Task Force (IETF), BCP 225, feb 2020. (Citato a pagina 16)
- [9] D. Temoshok, D. Proud-Madruga, Y.-Y. Choong, R. Galluzzo, S. Gupta, C. LaSalle, N. Lefkovitz, and A. Regenscheid, “Digital identity guidelines: Authentication and authenticator management,” National Institute of Standards and Technology, NIST Special Publication 800-63B-4, jul 2025. (Citato alle pagine 16 e 21)
- [10] OWASP Foundation, “Authentication Cheat Sheet,” https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html, oWASP Cheat Sheet Series. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html (Citato a pagina 16)
- [11] J. Yang, E. Wang, J. Chen, Q. Wang, Y. Zhang, H. Duan, W. Xie, and B. Wang, “Token Time Bomb: Evaluating JWT Implementations for Vulnerability Discove-

- ry,” in *Proceedings of the 32nd Network and Distributed System Security Symposium (NDSS)*. Internet Society, 2025. (Citato a pagina 16)
- [12] B. Xu, S. Jia, J. Lin, F. Zheng, Y. Ma, L. Liu, X. Gu, and L. Song, “JWTKey: Automatic Cryptographic Vulnerability Detection in JWT Applications,” in *Computer Security – ESORICS 2023*, ser. Lecture Notes in Computer Science, vol. 14346. Cham: Springer, 2023, pp. 263–282. (Citato a pagina 16)
- [13] OWASP API Security Project, “API4:2023 Unrestricted Resource Consumption,” <https://owasp.org/API-Security/editions/2023/en/0xa4-unrestricted-resource-consumption/>, 2023, consultato il 25 agosto 2026. [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0xa4-unrestricted-resource-consumption/> (Citato alle pagine 20, 21 e 61)
- [14] A. Cha, E. Wittern, G. Baudart, J. C. Davis, L. Mandel, and J. A. Laredo, “A principled approach to GraphQL query cost analysis,” in *Proceedings of the 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE 2020)*. ACM, 2020, pp. 257–268. (Citato alle pagine 20 e 21)
- [15] S. A. Crosby and D. S. Wallach, “Denial of service via algorithmic complexity attacks,” in *Proceedings of the 12th USENIX Security Symposium*. Washington, DC: USENIX Association, 2003, pp. 29–44. (Citato a pagina 20)
- [16] C.-A. Staicu and M. Pradel, “Freezing the web: A study of ReDoS vulnerabilities in JavaScript-based web servers,” in *Proceedings of the 27th USENIX Security Symposium (USENIX Security 18)*. Baltimore, MD: USENIX Association, 2018, pp. 361–376. (Citato alle pagine 20 e 21)
- [17] MITRE Corporation, “CWE-770: Allocation of resources without limits or throttling,” <https://cwe.mitre.org/data/definitions/770.html>, 2026, cWE versione 4.20. Consultato il 25 agosto 2026. [Online]. Available: <https://cwe.mitre.org/data/definitions/770.html> (Citato alle pagine 21, 22 e 62)
- [18] M. Nottingham and R. T. Fielding, “Additional HTTP status codes,” Internet Engineering Task Force, RFC 6585, apr 2012. (Citato a pagina 21)

- [19] R. Polli, A. Martinez Ruiz, and D. Miller, "RateLimit header fields for HTTP," IETF HTTPAPI Working Group, Internet-Draft draft-ietf-httpapi-ratelimit-headers-11, may 2026, work in progress. <https://datatracker.ietf.org/doc/draft-ietf-httpapi-ratelimit-headers/>. (Citato a pagina 21)
- [20] R. Chandramouli, "Security strategies for microservices-based application systems," National Institute of Standards and Technology, NIST Special Publication 800-204, aug 2019. (Citato a pagina 21)
- [21] J. Antunes, N. F. Neves, and P. Veríssimo, "Detection and prediction of resource-exhaustion vulnerabilities," in *Proceedings of the 19th IEEE International Symposium on Software Reliability Engineering (ISSRE 2008)*. IEEE, 2008. (Citato a pagina 22)
- [22] OWASP API Security Project, "API9:2023 Improper Inventory Management," <https://owasp.org/API-Security/editions/2023/en/0xa9-improper-inventory-management/>, 2023, consultato il 25 agosto 2026. [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0xa9-improper-inventory-management/> (Citato alle pagine 22, 24, 31, 32 e 33)
- [23] —, "API5:2023 broken function level authorization," <https://owasp.org/API-Security/editions/2023/en/0xa5-broken-function-level-authorization/>, 2023, oWASP API Security Top 10 – 2023. Consultato il 25 agosto 2026. [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0xa5-broken-function-level-authorization/> (Citato alle pagine 22, 23, 24 e 65)
- [24] MITRE Corporation, "CWE-285: Improper Authorization," <https://cwe.mitre.org/data/definitions/285.html>, common Weakness Enumeration. [Online]. Available: <https://cwe.mitre.org/data/definitions/285.html> (Citato alle pagine 23 e 66)
- [25] OWASP API Security Project, "API8:2023 security misconfiguration," <https://owasp.org/API-Security/editions/2023/en/0xa8-security-misconfiguration/>, 2023, oWASP API Security Top 10 – 2023. Consultato il 25 agosto 2026. [Online]. Available: <https://owasp.org/>

- API-Security/editions/2023/en/0xa8-security-misconfiguration/ (Citato alle pagine 24, 29, 30, 31 e 72)
- [26] F. Sun, L. Xu, and Z. Su, "Static detection of access control vulnerabilities in web applications," in *Proceedings of the 20th USENIX Security Symposium (USENIX Security 11)*. San Francisco, CA: USENIX Association, 2011. (Citato a pagina 24)
- [27] S. Son, K. S. McKinley, and V. Shmatikov, "RoleCast: Finding missing security checks when you do not know what checks are," in *Proceedings of the 2011 ACM International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA 2011)*. ACM, 2011, pp. 1069–1084. (Citato a pagina 25)
- [28] M. Monshizadeh, P. Naldurg, and V. N. Venkatakrishnan, "MACE: Detecting privilege escalation vulnerabilities in web applications," in *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security (CCS 2014)*. ACM, 2014, pp. 690–701. (Citato a pagina 25)
- [29] V. Atlidakis, P. Godefroid, and M. Polishchuk, "Checking security properties of cloud service REST APIs," in *Proceedings of the 13th IEEE International Conference on Software Testing, Validation and Verification (ICST 2020)*. IEEE, 2020, pp. 387–397. (Citato a pagina 25)
- [30] F. Liu, Y. Zhang, E. Li, W. Meng, Y. Shi, Q. Wang, C. Wang, Z. Lin, and M. Yang, "BACScan: Automatic black-box detection of broken-access-control vulnerabilities in web applications," in *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS 2025)*. ACM, 2025, pp. 1320–1333. (Citato a pagina 25)
- [31] OWASP API Security Project, "API6:2023 Unrestricted Access to Sensitive Business Flows," <https://owasp.org/API-Security/editions/2023/en/0xa6-unrestricted-access-to-sensitive-business-flows/>, 2023, consultato il 25 agosto 2026. [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0xa6-unrestricted-access-to-sensitive-business-flows/> (Citato alle pagine 25, 26 e 37)

-
- [32] V. Felmetsger, L. Cavedon, C. Kruegel, and G. Vigna, "Toward automated detection of logic vulnerabilities in web applications," in *Proceedings of the 19th USENIX Security Symposium (USENIX Security 10)*. Washington, DC: USENIX Association, 2010, pp. 143–160. (Citato alle pagine 26 e 36)
- [33] G. Pellegrino and D. Balzarotti, "Toward black-box detection of logic flaws in web applications," in *Proceedings of the 21st Network and Distributed System Security Symposium (NDSS 2014)*, San Diego, CA, 2014. (Citato alle pagine 26 e 36)
- [34] OWASP API Security Project, "API7:2023 server side request forgery," <https://owasp.org/API-Security/editions/2023/en/0xa7-server-side-request-forgery/>, 2023, oWASP API Security Top 10 – 2023. Consultato il 25 agosto 2026. [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0xa7-server-side-request-forgery/> (Citato alle pagine 27 e 28)
- [35] MITRE Corporation, "CWE-918: Server-side request forgery (SSRF)," <https://cwe.mitre.org/data/definitions/918.html>, 2026, cWE versione 4.20. Consultato il 25 agosto 2026. [Online]. Available: <https://cwe.mitre.org/data/definitions/918.html> (Citato alle pagine 27 e 28)
- [36] M. Wessels, S. Koch, G. Pellegrino, and M. Johns, "SSRF vs. developers: A study of SSRF-defenses in PHP applications," in *Proceedings of the 33rd USENIX Security Symposium (USENIX Security 24)*. Philadelphia, PA: USENIX Association, 2024, pp. 6777–6794. (Citato a pagina 28)
- [37] Y. Ji, T. Dai, Z. Zhou, Y. Tang, and J. He, "Artemis: Toward accurate detection of server-side request forgeries through LLM-assisted inter-procedural path-sensitive taint analysis," *Proceedings of the ACM on Programming Languages*, 2025, preprint disponibile come arXiv:2502.21026. (Citato a pagina 28)
- [38] E. Wang *et al.*, "Where URLs become weapons: Automated discovery of SSRF vulnerabilities in web applications," in *Proceedings of the 45th IEEE Symposium on Security and Privacy (S&P 2024)*. IEEE, 2024. (Citato a pagina 28)

- [39] MITRE Corporation, “CWE-16: Configuration,” <https://cwe.mitre.org/data/definitions/16.html>, 2026, categoria CWE, versione 4.20; mappatura vietata. Consultato il 25 agosto 2026. [Online]. Available: <https://cwe.mitre.org/data/definitions/16.html> (Citato a pagina 29)
- [40] —, “CWE-1059: Insufficient technical documentation,” <https://cwe.mitre.org/data/definitions/1059.html>, 2026, cWE versione 4.20; denominazione precedente *Incomplete Documentation* fino ad aprile 2022; mappatura vietata. Consultato il 25 agosto 2026. [Online]. Available: <https://cwe.mitre.org/data/definitions/1059.html> (Citato a pagina 31)
- [41] OWASP API Security Project, “Release notes – OWASP API security top 10 2023,” <https://owasp.org/API-Security/editions/2023/en/0x04-release-notes/>, 2023, consultato il 25 agosto 2026. [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0x04-release-notes/> (Citato a pagina 33)
- [42] —, “API10:2023 Unsafe Consumption of APIs,” <https://owasp.org/API-Security/editions/2023/en/0xaa-unsafe-consumption-of-apis/>, 2023, consultato il 25 agosto 2026. [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0xaa-unsafe-consumption-of-apis/> (Citato alle pagine 34, 35 e 74)
- [43] OWASP Foundation, “crAPI: completely ridiculous API,” <https://github.com/OWASP/crAPI>, 2026. [Online]. Available: <https://github.com/OWASP/crAPI> (Citato alle pagine 40, 49 e 55)
- [44] Prisma Cloud, “Checkov – what is checkov?” <https://www.checkov.io/>, 2026, analizzatore statico per Infrastructure-as-Code. [Online]. Available: <https://www.checkov.io/> (Citato a pagina 44)
- [45] HashiCorp, “Terraform documentation,” <https://developer.hashicorp.com/terraform/docs>, 2026. [Online]. Available: <https://developer.hashicorp.com/terraform/docs> (Citato a pagina 44)

-
- [46] M. Brunsfeld *et al.*, “Tree-sitter: an incremental parsing system for programming tools,” <https://tree-sitter.github.io/tree-sitter/>, 2026. [Online]. Available: <https://tree-sitter.github.io/tree-sitter/> (Citato alle pagine 52, 58 e 61)
- [47] OWASP API Security Project, “API3:2023 Broken Object Property Level Authorization,” <https://owasp.org/API-Security/editions/2023/en/0xa3-broken-object-property-level-authorization/>, 2023, consultato il 25 agosto 2026. [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0xa3-broken-object-property-level-authorization/> (Citato a pagina 56)
- [48] MITRE Corporation, “CWE-400: Uncontrolled resource consumption,” <https://cwe.mitre.org/data/definitions/400.html>, 2026, consultato il 25 agosto 2026. [Online]. Available: <https://cwe.mitre.org/data/definitions/400.html> (Citato a pagina 62)
- [49] —, “CWE-799: Improper control of interaction frequency,” <https://cwe.mitre.org/data/definitions/799.html>, 2026, consultato il 25 agosto 2026. [Online]. Available: <https://cwe.mitre.org/data/definitions/799.html> (Citato a pagina 62)
- [50] —, “CWE-862: Missing Authorization,” <https://cwe.mitre.org/data/definitions/862.html>, common Weakness Enumeration. [Online]. Available: <https://cwe.mitre.org/data/definitions/862.html> (Citato a pagina 66)
- [51] —, “CWE-650: Trusting HTTP Permission Methods on the Server Side,” <https://cwe.mitre.org/data/definitions/650.html>, common Weakness Enumeration. [Online]. Available: <https://cwe.mitre.org/data/definitions/650.html> (Citato a pagina 66)
- [52] —, “CWE-284: Improper Access Control,” <https://cwe.mitre.org/data/definitions/284.html>, common Weakness Enumeration. [Online]. Available: <https://cwe.mitre.org/data/definitions/284.html> (Citato a pagina 66)
- [53] —, “CWE-276: Incorrect Default Permissions,” <https://cwe.mitre.org/data/definitions/276.html>, common Weakness Enumeration. [Online]. Available: <https://cwe.mitre.org/data/definitions/276.html> (Citato a pagina 66)

- [54] OWASP API Security Project, “API10:2023 unsafe consumption of APIs – conflicting vulnerability indicators,” <https://github.com/OWASP/API-Security/issues/80>, 2023, issue n. 80 del repository GitHub OWASP/API-Security, marzo 2023. Consultato il 25 agosto 2026. [Online]. Available: <https://github.com/OWASP/API-Security/issues/80> (Citato a pagina 74)
- [55] MITRE Corporation, “CWE-20: Improper Input Validation,” <https://cwe.mitre.org/data/definitions/20.html>, common Weakness Enumeration. [Online]. Available: <https://cwe.mitre.org/data/definitions/20.html> (Citato a pagina 75)
- [56] —, “CWE-319: Cleartext Transmission of Sensitive Information,” <https://cwe.mitre.org/data/definitions/319.html>, common Weakness Enumeration. [Online]. Available: <https://cwe.mitre.org/data/definitions/319.html> (Citato a pagina 75)
- [57] —, “CWE-601: URL Redirection to Untrusted Site (Open Redirect),” <https://cwe.mitre.org/data/definitions/601.html>, common Weakness Enumeration. [Online]. Available: <https://cwe.mitre.org/data/definitions/601.html> (Citato a pagina 75)

Ringraziamenti

Ringraziamenti qui...